

Mano Vektorius

Generated by Doxygen 1.17.0

1 README	1
1.0.1 V0.4 testavimai	1
1.0.2 Padaryti pokyčiai - Struct pakeistas į Class.	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 MyVector< T > Class Template Reference	9
5.1.1 Detailed Description	11
5.1.2 Member Typedef Documentation	11
5.1.2.1 value_type	11
5.1.3 Constructor & Destructor Documentation	11
5.1.3.1 MyVector() [1/3]	11
5.1.3.2 MyVector() [2/3]	11
5.1.3.3 MyVector() [3/3]	11
5.1.3.4 ~MyVector()	12
5.1.4 Member Function Documentation	12
5.1.4.1 assign()	12
5.1.4.2 at() [1/2]	12
5.1.4.3 at() [2/2]	12
5.1.4.4 back() [1/2]	13
5.1.4.5 back() [2/2]	13
5.1.4.6 begin() [1/2]	13
5.1.4.7 begin() [2/2]	13
5.1.4.8 capacity() [1/2]	14
5.1.4.9 capacity() [2/2]	14
5.1.4.10 cbegin()	14
5.1.4.11 cend()	14
5.1.4.12 clear()	14
5.1.4.13 data()	14
5.1.4.14 empty() [1/2]	15
5.1.4.15 empty() [2/2]	15
5.1.4.16 end() [1/2]	15
5.1.4.17 end() [2/2]	15
5.1.4.18 erase() [1/2]	15
5.1.4.19 erase() [2/2]	16
5.1.4.20 front() [1/2]	16

5.1.4.21 front() [2/2]	16
5.1.4.22 grow()	16
5.1.4.23 insert()	16
5.1.4.24 operator=() [1/2]	17
5.1.4.25 operator=() [2/2]	17
5.1.4.26 operator[]() [1/2]	17
5.1.4.27 operator[]() [2/2]	18
5.1.4.28 pop_back()	18
5.1.4.29 push_back()	18
5.1.4.30 reserve()	18
5.1.4.31 resize() [1/2]	18
5.1.4.32 resize() [2/2]	19
5.1.4.33 shrink_to_fit()	19
5.1.4.34 size() [1/2]	19
5.1.4.35 size() [2/2]	19
5.1.4.36 swap()	19
5.1.5 Member Data Documentation	20
5.1.5.1 capacity_	20
5.1.5.2 data_	20
5.1.5.3 size_	20
5.2 Studentas< GradeContainer > Class Template Reference	20
5.2.1 Detailed Description	21
5.2.2 Constructor & Destructor Documentation	21
5.2.2.1 Studentas() [1/3]	21
5.2.2.2 Studentas() [2/3]	21
5.2.2.3 Studentas() [3/3]	21
5.2.2.4 ~Studentas()	21
5.2.3 Member Function Documentation	22
5.2.3.1 egzaminas()	22
5.2.3.2 mediana()	22
5.2.3.3 operator=() [1/2]	22
5.2.3.4 operator=() [2/2]	22
5.2.3.5 pazymiai() [1/2]	22
5.2.3.6 pazymiai() [2/2]	22
5.2.3.7 setEgzaminas()	22
5.2.3.8 setMediana()	22
5.2.3.9 setVidurkis()	22
5.2.3.10 tipas()	23
5.2.3.11 vidurkis()	23
5.2.4 Member Data Documentation	23
5.2.4.1 egzaminas_	23
5.2.4.2 mediana_	23

5.2.4.3 pazymiai_	23
5.2.4.4 vidurkis_	23
5.3 TestStudent Class Reference	23
5.3.1 Detailed Description	24
5.3.2 Constructor & Destructor Documentation	24
5.3.2.1 TestStudent() [1/3]	24
5.3.2.2 TestStudent() [2/3]	24
5.3.2.3 TestStudent() [3/3]	24
5.3.3 Member Function Documentation	24
5.3.3.1 egzaminas()	24
5.3.3.2 mediana()	24
5.3.3.3 operator=() [1/2]	25
5.3.3.4 operator=() [2/2]	25
5.3.3.5 pavarde()	25
5.3.3.6 pazymiai() [1/2]	25
5.3.3.7 pazymiai() [2/2]	25
5.3.3.8 setEgzaminas()	25
5.3.3.9 setMediana()	25
5.3.3.10 setPavarde()	25
5.3.3.11 setVardas()	25
5.3.3.12 setVidurkis()	25
5.3.3.13 vardas()	25
5.3.3.14 vidurkis()	26
5.3.4 Member Data Documentation	26
5.3.4.1 egzaminas_	26
5.3.4.2 mediana_	26
5.3.4.3 pavarde_	26
5.3.4.4 pazymiai_	26
5.3.4.5 vardas_	26
5.3.4.6 vidurkis_	26
5.4 Zmogus Class Reference	26
5.4.1 Detailed Description	27
5.4.2 Constructor & Destructor Documentation	27
5.4.2.1 Zmogus() [1/3]	27
5.4.2.2 Zmogus() [2/3]	27
5.4.2.3 Zmogus() [3/3]	27
5.4.2.4 ~Zmogus()	27
5.4.3 Member Function Documentation	27
5.4.3.1 operator=() [1/2]	27
5.4.3.2 operator=() [2/2]	27
5.4.3.3 pavarde()	27
5.4.3.4 setPavarde()	27

5.4.3.5 setVardas()	28
5.4.3.6 vardas()	28
5.4.4 Member Data Documentation	28
5.4.4.1 pavarde_	28
5.4.4.2 vardas_	28
6 File Documentation	29
6.1 funkcijos.h	29
6.2 library.h	38
6.3 main.cpp	38
6.4 objektinis_patikrinimai.cpp	39
6.5 patikrinimai.h	40
6.6 rule_of_five_test.cpp	40
6.7 vector.h	46
Index	51

Chapter 1

README

Kaip naudotis programa:

1. make clean - išvalyti kompiliacijos failus.
2. make - komanda sukuria failą "programa" pagal makefile.
3. ./programa - komanda paleidžia pačią programą.

Papildoma: Norint atlikti programos testavimą konsolėje reikia įrašyti komandą - make test.

1.0.1 V0.4 testavimai

4 versioj duomenų įvedimo sistemoj buvo pridėta nauja failų generavimo funkcija, kuri kuria failus pagal šablonus naudotis 2 versioj. Buvo atlikti testavimai laiko apskaičiavimui ir jų rezultatai bus pateikti žemiau: 1 testavimas: Duoti testavimai turi skirtingus studentų pažymių kiekius, kad matytusi skirtumas tarp atlikimo laikų.

2 testavimas (visi testai buvo atlikti su pažymių sk. = 2 ir rušiuoti pagal medianą):

V1.0 testavimai Sistema: CPU - Apple M2 (8 branduoliai) RAM - 16gb SSD - 256gb

1 strategija - Duomenys nuskaityti į vieną duomenų konteinerį ir skirstymo metu masyvas skirstomas į maladiec ir lopų konteinerius. Rezultatai:

2 strategija - Duomenys nuskaityti į "maladiec" konteinerį ir žmones su mažesniu vidurkiu yra įdedami į lopai konteineri ir iškerpami. Rezultatai:

3 strategija - Padaryta pagal 2 strategija, tik vietoj erase yra naudojamas partition algoritmas. Rezultatai:

V1.1 TESTAVIMAI

1.0.2 Padaryti pokyčiai - Struct pakeistas į Class.

Greičio patikrinimas po pokyčių: Aiškių skirtumų tarp atlikimo laiko nėra.

Optimizacijos Flagai. Testavimui buvo naudojami optimizacijos flagai -O1, -O2, -O3 ir testavimai buvo atlikti su tais pačiais failais.

-O1:

"Programa" paleidimo failo dydis naudojant pirmą flagą - 330Kb.

-O2:

"Programa" paleidimo failo dydis naudojant pirmą flagą - 314Kb.

-O3:

"Programa" paleidimo failo dydis naudojant pirmą flagą - 329Kb.

Išvados: Matosi aiškus skirtumas tarp programų atlikimo laiko kur nebuvo naudojami optimizacijos flagai ir kur buvo, kadangi skirtumas atlikimo laike vos ne dvigubai skiriasi, bet atlikimo greičio skirtumas tarp pačių flagų nėra aiškiai pastebimas.

V1.2 TESTAVIMAI Padaryti pokyčiai - įgyvendinta "Rule of five" ir įvesties/išvesties operatoriai. Taip pat pridėtas testavimo metodas, kuris patikrina, kad visi metodai veikia.

Apie išvesties ir įvesties metodus:

1. `operator>>` nuskaityto vardą, pavardę, egzamino pažymį, pažymių kiekį ir pačius pažymius
2. `operator<<` išveda studento duomenis tuo pačiu formatu į failą arba į konsolę pagal vartotojo pasirinkimą.
3. šie operatoriai buvo testuojami [rule_of_five_test.cpp](#)

Kaip veikia testavimas: Testavimo kodas yra paleidžiamas konsolėje įvedus `./rule_of_five_test`. Jis naudojamas visų naujų metodų patikrai. Kodui praėjus testavimą į konsolę yra išvedama eilutė "Visi rule of five ir operatoriu testai praėjo.", jeigu programa yra terminuojama, reiškias program nepraejo testavimų.

V1.5 TESTAVIMAI

Padaryti pokyčiai - sukurta abstrakti klasė "Zmogus", kurios išvestinė yra programos pagrindinė naudojama klasė "Studentas". Šioje klasėje yra saugomi tokie parametrai kaip vardas ir pavardė, kuriuos paveldi klasė "Studentas". "Zmogus" klasės objektus sukurti nėra įmanoma, kadangi čia yra abstrakti klasė, o jos išvestinių klasių, kaip "← Studentas", objektų kūrimas yra įmanomas.

Ši nauja versija palaiko visas prieš tai realizuotas funkcijas.

V2.0 UNIT TESTAI Realizuoti paprasti unit testai faile [rule_of_five_test.cpp](#). Testuose naudojama pagalbinė check funkcija, kuri patikrina sąlygą ir išveda klaidos pranešimą.

Testai paleidžiami komanda:

```
make test
```

Pateikti testai tikrina:

```
5-ių metodų taisyklę: kopijavimo konstruktorių, perkėlimo konstruktorių, kopijavimo priskyrimą, perkėlimo pris-
Studentas veikimą per abstrakčią Zmogus klasės sąsają.
Įvesties ir išvesties operatorius >> ir <<.
Vardo bei skaičių validacijos funkcijas.
Galutinio vidurkio ir medianos skaičiavimą.
```

V3.0 TESTAVIMAI

1.0; Padaryti pokyčiai - sukurtas naujas konteineris [MyVector](#), kuris yra naudojamas vietoj standartinio `std::vector`. Šis konteineris ir jo visos funkcijos yra aprašytos faile "vector.cpp", jis įgyvendina 80% standartinio vektoriaus funkcijų.

Pačią vektoriaus klasę sudaro tik trys kintamieji. `Size_` - elementų kiekis vektoriuje `Capacity_` kiek vietos yra dedikuota vektoriumi `data_` saugomi duomenys vektoriuje

Sukurtų funkcijų pavyzdžiai:

`Grow()`: `Grow` yra privati funkcija, kurios negalima iškviešti už pačios klasės ribų. Jos paskirtis yra didinti vektoriaus talpą automatiškai, kai rašymo metu yra pasiekiamas limitas. //nuotrauka funkcijos `grow`.

`push_back()`: `Push_back` funkcija yra naudojama naujam elementui įrašyti į vektoriaus galą. Prieš įrašant reikšmę yra patikrinama ar vektoriuje dar yra laisvos vietos. Jeigu `size_` yra lygus `capacity_`, tada iškviečiama `grow()` funkcija, kuri padidina vektoriaus talpą. Po to nauja reikšmė yra įrašoma į `data_[size_]` vietą ir `size_` padidinamas vienetu. //nuotrauka funkcijos `push_back`.

`pop_back()`: `Pop_back` funkcija pašalina paskutinį vektoriaus elementą. Funkcija pirmiausia patikrina ar vektorius nėra tuščias, tai yra ar `size_` yra daugiau už 0. Jeigu vektoriuje yra elementų, `size_` yra sumažinamas vienetu. Pats elementas iš atminties nėra fiziškai ištrinamas, bet jis tampa nebenaudojamas, nes vektoriaus dydis sumažėja. //nuotrauka funkcijos `pop_back`.

`reserve()`: `Reserve` funkcija yra naudojama iš anksto padidinti vektoriaus talpą. Ji nekeičia `size_`, todėl vektoriaus elementų kiekis lieka toks pats. Jeigu vartotojo nurodytas naujas `capacity` yra mažesnis arba lygus dabartiniam `capacity_`, funkcija nieko nedaro. Jeigu naujas `capacity` yra didesnis, sukuriamas naujas masyvas, į jį perkeliama seni duomenys, senas masyvas ištrinamas ir `data_` pradeda rodyti į naują masyvą. //nuotrauka funkcijos `reserve`.

`erase()`: `Erase` funkcija yra naudojama pašalinti vieną elementą arba elementų intervalą iš vektoriaus. Funkcijai perduodami iteratoriai, kurie nurodo nuo kurios vietos iki kurios vietos reikia trinti elementus. Po pašalinimo likę elementai yra perstumiami į kairę, kad vektoriuje neliktų tuščių tarpų. Galiausiai `size_` yra sumažinamas pagal pašalintų elementų kiekį. //nuotrauka funkcijos `erase`. 2.0:

3.0: Atminties perskirstymas užpildant `std::vector` naudojant studentų failą su 10000000 studentų duomenimis įvyko 25 kartus, o užpildant mano sukurtą vektorius atminties perskirstymas įvyko 24 kartus.

4.0: Spartos testavimai naudojant studentų failus 1000 - 10000000 dydžių su `std::vector` ir mano vektorium:

Pagal atliktus testavimus matome, kad mano vektorius skaitymą iš failo atlieka lėčiau negu `std::vector`, bet `sort` ir skirstymo funkcijas yra atliekamos arba panašiu greičiu, arba greičiau už `std::vector`.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

MyVector< T >	9
TestStudent	23
Zmogus	26
Studentas< std::vector< int > >	20
Studentas< std::list< int > >	20
Studentas< std::deque< int > >	20
Studentas< MyVector< int > >	20
Studentas< GradeContainer >	20

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

MyVector< T >	
Papras tas dinaminio masyvo konteineris, panasus i std::vector	9
Studentas< GradeContainer >	20
TestStudent	23
Zmogus	26

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

funkcijos.h	29
library.h	38
main.cpp	38
objektinis_patikrinimai.cpp	39
patikrinimai.h	40
rule_of_five_test.cpp	40
vector.h	46

Chapter 5

Class Documentation

5.1 `MyVector< T >` Class Template Reference

Paprastas dinaminio masyvo konteineris, panašus į `std::vector`.

```
#include <vector.h>
```

Public Types

- using `value_type` = `T`
Konteineryje saugomu elementu tipas.

Public Member Functions

- `MyVector` ()
Sukuria tuscia konteineri su pradine talpa 1.
- `MyVector` (const `MyVector` &other)
Kopijavimo konstruktorius.
- `MyVector` (`MyVector` &&other)
Perkelimo konstruktorius.
- `T` & `operator[]` (size_t index)
Grazina elemento nuoroda pagal indeksa be ribu tikrinimo.
- `MyVector` & `operator=` (const `MyVector` &other)
Kopijavimo priskyrimo operatorius.
- `MyVector` & `operator=` (`MyVector` &&other)
Perkelimo priskyrimo operatorius.
- const `T` & `operator[]` (size_t index) const
Grazina elemento konstancia nuoroda pagal indeksa be ribu tikrinimo.
- void `push_back` (const `T` &val)
Prideda elementa i konteinerio gala.
- void `pop_back` ()
Pasalina paskutini elementa, jei konteineris netuscias.
- bool `empty` ()
Patikrina, ar konteineris tuscias.
- bool `empty` () const
Patikrina, ar konteineris tuscias.
- size_t `size` ()
Grazina elementu kieki.
- size_t `size` () const
Grazina elementu kieki.
- size_t `capacity` ()

- Grazina dabartine konteinerio talpa.*
- `size_t capacity () const`
- Grazina dabartine konteinerio talpa.*
- `void clear ()`
- Pasalina visus elementus, bet talpos nemazina.*
- `void shrink_to_fit ()`
- Sumazina talpa iki dabartinio elementu kiekio.*
- `T * begin ()`
- Grazina iteratoriu i pirma elementa.*
- `T * end ()`
- Grazina iteratoriu uz paskutinio elemento.*
- `const T * begin () const`
- Grazina konstantu iteratoriu i pirma elementa.*
- `const T * end () const`
- Grazina konstantu iteratoriu uz paskutinio elemento.*
- `T & back ()`
- Grazina paskutinio elemento nuoroda.*
- `const T & back () const`
- Grazina paskutinio elemento konstancia nuoroda.*
- `T & front ()`
- Grazina pirmo elemento nuoroda.*
- `const T & front () const`
- Grazina pirmo elemento konstancia nuoroda.*
- `T * erase (T *first, T *last)`
- Pasalina elementu intervala.*
- `T * erase (T *pos)`
- Pasalina viena elementa.*
- `T & at (size_t index)`
- Grazina elemento nuoroda su ribu tikrinimu.*
- `const T & at (size_t index) const`
- Grazina konstancia elemento nuoroda su ribu tikrinimu.*
- `void reserve (size_t newCapacity)`
- Padidina konteinerio talpa, jei nauja talpa didesne uz esama.*
- `void resize (size_t newSize)`
- Pakeicia konteinerio dydi.*
- `void resize (size_t newSize, const T &value)`
- Pakeicia konteinerio dydi ir naujus elementus uzpildo reiksme.*
- `T * data ()`
- Grazina rodykle i vidini masyva.*
- `T * cbegin () const`
- Grazina konstantu iteratoriu i pradzia.*
- `const T * cend () const`
- Grazina konstantu iteratoriu uz paskutinio elemento.*
- `void swap (MyVector &other)`
- Apkeicia dviejų konteineriu duomenis.*
- `void assign (size_t count, const T &value)`
- Uzpildo konteineri nurodytu kiekiu vienodu reiksniu.*
- `T * insert (T *pos, const T &value)`
- Iterpia elementa nurodytoje vietoje.*
- `~MyVector ()`
- Atlaisvina konteinerio naudojama atminti.*

Private Member Functions

- void [grow](#) ()
Padidina konteinerio talpa dvigubai.

Private Attributes

- size_t [size_](#)
- size_t [capacity_](#)
- T * [data_](#)

5.1.1 Detailed Description

template<typename T>
class MyVector< T >

Paprastas dinaminio masyvo konteineris, panasus i std::vector.

Template Parameters

<i>T</i>	Saugomu elementu tipas.
----------	-------------------------

Definition at line 12 of file [vector.h](#).

5.1.2 Member Typedef Documentation

5.1.2.1 value_type

```
template<typename T>
using MyVector< T >::value_type = T
```

Konteineryje saugomu elementu tipas.
 Definition at line 35 of file [vector.h](#).

5.1.3 Constructor & Destructor Documentation

5.1.3.1 MyVector() [1/3]

```
template<typename T>
MyVector< T >::MyVector () [inline]
```

Sukuria tuscia konteineri su pradine talpa 1.
 Definition at line 44 of file [vector.h](#).

5.1.3.2 MyVector() [2/3]

```
template<typename T>
MyVector< T >::MyVector (
    const MyVector< T > & other) [inline]
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	Konteineris, is kurio kopijuojami duomenys.
--------------	---

Definition at line 53 of file [vector.h](#).

5.1.3.3 MyVector() [3/3]

```
template<typename T>
MyVector< T >::MyVector (
    MyVector< T > && other) [inline]
```

Perkelimo konstruktorius.

Parameters

<i>other</i>	Konteineris, kurio duomenys perkeliama.
--------------	---

Definition at line 66 of file [vector.h](#).

5.1.3.4 ~MyVector()

```
template<typename T>
MyVector< T >::~~MyVector () [inline]
```

Atlaisvina konteinerio naudojama atminti.

Definition at line 464 of file [vector.h](#).

5.1.4 Member Function Documentation

5.1.4.1 assign()

```
template<typename T>
void MyVector< T >::assign (
    size_t count,
    const T & value) [inline]
```

Uzpildo konteineri nurodytu kiekiu vienodu reiksmiu.

Parameters

<i>count</i>	Elementu kiekis.
<i>value</i>	Priskiriama reiksme.

Definition at line 432 of file [vector.h](#).

5.1.4.2 at() [1/2]

```
template<typename T>
T & MyVector< T >::at (
    size_t index) [inline]
```

Grazina elemento nuoroda su ribu tikrinimu.

Parameters

<i>index</i>	Elemento indeksas.
--------------	--------------------

Returns

Elemento nuoroda.

Exceptions

<i>std::out_of_range</i>	Jei indeksas uz konteinerio ribu.
--------------------------	-----------------------------------

Definition at line 324 of file [vector.h](#).

5.1.4.3 at() [2/2]

```
template<typename T>
const T & MyVector< T >::at (
```

```
size_t index) const [inline]
```

Grazina konstancia elemento nuoroda su ribu tikrinimu.

Parameters

<i>index</i>	Elemento indeksas.
--------------	--------------------

Returns

Konstanti elemento nuoroda.

Exceptions

<i>std::out_of_range</i>	Jei indeksas uz konteinerio ribu.
--------------------------	-----------------------------------

Definition at line 337 of file [vector.h](#).

5.1.4.4 back() [1/2]

```
template<typename T>
T & MyVector< T >::back () [inline]
```

Grazina paskutinio elemento nuoroda.

Returns

Paskutinio elemento nuoroda.

Definition at line 269 of file [vector.h](#).

5.1.4.5 back() [2/2]

```
template<typename T>
const T & MyVector< T >::back () const [inline]
```

Grazina paskutinio elemento konstancia nuoroda.

Returns

Konstanti paskutinio elemento nuoroda.

Definition at line 277 of file [vector.h](#).

5.1.4.6 begin() [1/2]

```
template<typename T>
T * MyVector< T >::begin () [inline]
```

Grazina iteratoriu i pirma elementa.

Returns

Rodykle i pirma elementa.

Definition at line 237 of file [vector.h](#).

5.1.4.7 begin() [2/2]

```
template<typename T>
const T * MyVector< T >::begin () const [inline]
```

Grazina konstantu iteratoriu i pirma elementa.

Returns

Konstanti rodykle i pirma elementa.

Definition at line 253 of file [vector.h](#).

5.1.4.8 capacity() [1/2]

```
template<typename T>
size_t MyVector< T >::capacity () [inline]
```

Grazina dabartine konteinerio talpa.

Returns

Talpa.

Definition at line 197 of file [vector.h](#).

5.1.4.9 capacity() [2/2]

```
template<typename T>
size_t MyVector< T >::capacity () const [inline]
```

Grazina dabartine konteinerio talpa.

Returns

Talpa.

Definition at line 205 of file [vector.h](#).

5.1.4.10 cbegin()

```
template<typename T>
T * MyVector< T >::cbegin () const [inline]
```

Grazina konstantu iteratoriu i pradzia.

Returns

Konstanti rodykle i duomenu pradzia.

Definition at line 405 of file [vector.h](#).

5.1.4.11 cend()

```
template<typename T>
const T * MyVector< T >::cend () const [inline]
```

Grazina konstantu iteratoriu uz paskutinio elemento.

Returns

Konstanti rodykle uz paskutinio elemento.

Definition at line 413 of file [vector.h](#).

5.1.4.12 clear()

```
template<typename T>
void MyVector< T >::clear () [inline]
```

Pasalina visus elementus, bet talpos nemazina.

Definition at line 212 of file [vector.h](#).

5.1.4.13 data()

```
template<typename T>
T * MyVector< T >::data () [inline]
```

Grazina rodykle i vidini masyva.

Returns

Rodykle i duomenu pradzia.

Definition at line 397 of file [vector.h](#).

5.1.4.14 empty() [1/2]

```
template<typename T>
bool MyVector< T >::empty () [inline]
```

Patikrina, ar konteineris tuscias.

Returns

true, jei elementu nera.

Definition at line 165 of file [vector.h](#).

5.1.4.15 empty() [2/2]

```
template<typename T>
bool MyVector< T >::empty () const [inline]
```

Patikrina, ar konteineris tuscias.

Returns

true, jei elementu nera.

Definition at line 173 of file [vector.h](#).

5.1.4.16 end() [1/2]

```
template<typename T>
T * MyVector< T >::end () [inline]
```

Grazina iteratoriu uz paskutinio elemento.

Returns

Rodykle uz paskutinio elemento.

Definition at line 245 of file [vector.h](#).

5.1.4.17 end() [2/2]

```
template<typename T>
const T * MyVector< T >::end () const [inline]
```

Grazina konstantu iteratoriu uz paskutinio elemento.

Returns

Konstanti rodykle uz paskutinio elemento.

Definition at line 261 of file [vector.h](#).

5.1.4.18 erase() [1/2]

```
template<typename T>
T * MyVector< T >::erase (
    T * first,
    T * last) [inline]
```

Pasalina elementu intervala.

Parameters

<i>first</i>	Pirmas salinamas elementas.
<i>last</i>	Iteratorius uz paskutinio salinamo elemento.

Returns

Iteratorius i vieta, kur buvo pasalintas intervalas.

Definition at line 303 of file [vector.h](#).

5.1.4.19 erase() [2/2]

```
template<typename T>
T * MyVector< T >::erase (
    T * pos) [inline]
```

Pasalina viena elementa.

Parameters

<i>pos</i>	Salinamo elemento iteratorius.
------------	--------------------------------

Returns

Iteratorius i vieta, kur buvo pasalintas elementas.

Definition at line 314 of file [vector.h](#).

5.1.4.20 front() [1/2]

```
template<typename T>
T & MyVector< T >::front () [inline]
```

Grazina pirmo elemento nuoroda.

Returns

Pirmo elemento nuoroda.

Definition at line 285 of file [vector.h](#).

5.1.4.21 front() [2/2]

```
template<typename T>
const T & MyVector< T >::front () const [inline]
```

Grazina pirmo elemento konstancia nuoroda.

Returns

Konstanti pirmo elemento nuoroda.

Definition at line 293 of file [vector.h](#).

5.1.4.22 grow()

```
template<typename T>
void MyVector< T >::grow () [inline], [private]
```

Padidina konteinerio talpa dvigubai.

Definition at line 20 of file [vector.h](#).

5.1.4.23 insert()

```
template<typename T>
T * MyVector< T >::insert (
    T * pos,
    const T & value) [inline]
```

Iterpia elementa nurodytoje vietoje.

Parameters

<i>pos</i>	Iterpimo vieta.
<i>value</i>	Iterpiama reiksme.

Returns

Iteratorius i iterpta elementa.

Definition at line 446 of file [vector.h](#).

5.1.4.24 operator=() [1/2]

```
template<typename T>
MyVector & MyVector< T >::operator= (
    const MyVector< T > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	Konteineris, is kurio kopijuojami duomenys.
--------------	---

Returns

Nuoroda i si konteineri.

Definition at line 95 of file [vector.h](#).

5.1.4.25 operator=() [2/2]

```
template<typename T>
MyVector & MyVector< T >::operator= (
    MyVector< T > && other) [inline]
```

Perkelimo priskyrimo operatorius.

Parameters

<i>other</i>	Konteineris, kurio duomenys perkeliama.
--------------	---

Returns

Nuoroda i si konteineri.

Definition at line 114 of file [vector.h](#).

5.1.4.26 operator[]() [1/2]

```
template<typename T>
T & MyVector< T >::operator[] (
    size_t index) [inline]
```

Grazina elemento nuoroda pagal indeksa be ribu tikrinimo.

Parameters

<i>index</i>	Elemento indeksas.
--------------	--------------------

Returns

Elemento nuoroda.

Definition at line 86 of file [vector.h](#).

5.1.4.27 operator[]() [2/2]

```
template<typename T>
const T & MyVector< T >::operator[] (
    size_t index) const [inline]
```

Grazina elemento konstancia nuoroda pagal indeksa be ribu tikrinimo.

Parameters

<i>index</i>	Elemento indeksas.
--------------	--------------------

Returns

Konstanti elemento nuoroda.

Definition at line 135 of file [vector.h](#).

5.1.4.28 pop_back()

```
template<typename T>
void MyVector< T >::pop_back () [inline]
Pasalina paskutini elementa, jei konteineris netuscias.
Definition at line 157 of file vector.h.
```

5.1.4.29 push_back()

```
template<typename T>
void MyVector< T >::push_back (
    const T & val) [inline]
Prideda elementa i konteinerio gala.
```

Parameters

<i>val</i>	Pridedama reiksme.
------------	--------------------

Definition at line 148 of file [vector.h](#).

5.1.4.30 reserve()

```
template<typename T>
void MyVector< T >::reserve (
    size_t newCapacity) [inline]
Padidina konteinerio talpa, jei nauja talpa didesne uz esama.
```

Parameters

<i>newCapacity</i>	Nauja pageidaujama talpa.
--------------------	---------------------------

Definition at line 349 of file [vector.h](#).

5.1.4.31 resize() [1/2]

```
template<typename T>
void MyVector< T >::resize (
    size_t newSize) [inline]
Pakeicia konteinerio dydi.
```


Parameters

<i>newSize</i>	Naujas elementu kiekis.
----------------	-------------------------

Definition at line 368 of file [vector.h](#).

5.1.4.32 resize() [2/2]

```
template<typename T>
void MyVector< T >::resize (
    size_t newSize,
    const T & value) [inline]
```

Pakeicia konteinerio dydi ir naujus elementus uzpildo reiksme.

Parameters

<i>newSize</i>	Naujas elementu kiekis.
<i>value</i>	Reiksme naujiems elementams.

Definition at line 383 of file [vector.h](#).

5.1.4.33 shrink_to_fit()

```
template<typename T>
void MyVector< T >::shrink_to_fit () [inline]
```

Sumazina talpa iki dabartinio elementu kiekio.

Definition at line 219 of file [vector.h](#).

5.1.4.34 size() [1/2]

```
template<typename T>
size_t MyVector< T >::size () [inline]
```

Grazina elementu kiekį.

Returns

Elementu kiekis.

Definition at line 181 of file [vector.h](#).

5.1.4.35 size() [2/2]

```
template<typename T>
size_t MyVector< T >::size () const [inline]
```

Grazina elementu kiekį.

Returns

Elementu kiekis.

Definition at line 189 of file [vector.h](#).

5.1.4.36 swap()

```
template<typename T>
void MyVector< T >::swap (
    MyVector< T > & other) [inline]
```

Apkeicia dviejų konteinerių duomenis.

Parameters

<i>other</i>	Kitas konteineris.
--------------	--------------------

Definition at line 421 of file [vector.h](#).

5.1.5 Member Data Documentation**5.1.5.1 capacity_**

```
template<typename T>
size_t MyVector< T >::capacity_ [private]
```

Definition at line 15 of file [vector.h](#).

5.1.5.2 data_

```
template<typename T>
T* MyVector< T >::data_ [private]
```

Definition at line 16 of file [vector.h](#).

5.1.5.3 size_

```
template<typename T>
size_t MyVector< T >::size_ [private]
```

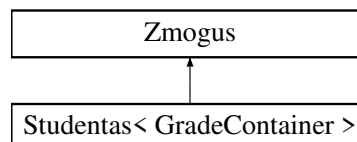
Definition at line 14 of file [vector.h](#).

The documentation for this class was generated from the following file:

- [vector.h](#)

5.2 Studentas< GradeContainer > Class Template Reference

Inheritance diagram for Studentas< GradeContainer >:

**Public Member Functions**

- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other)
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other)
- GradeContainer & [pazymiai](#) ()
- const GradeContainer & [pazymiai](#) () const
- int [egzaminas](#) () const
- void [setEgzaminas](#) (int egzaminas)
- double [vidurkis](#) () const
- void [setVidurkis](#) (double vidurkis)
- double [mediana](#) () const
- void [setMediana](#) (double mediana)
- std::string [tipas](#) () const override

Public Member Functions inherited from Zmogus

- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- [Zmogus](#) (const [Zmogus](#) &other)
- [Zmogus](#) ([Zmogus](#) &&other)
- [Zmogus](#) & operator= (const [Zmogus](#) &other)
- [Zmogus](#) & operator= ([Zmogus](#) &&other)
- const std::string & [vardas](#) () const
- void [setVardas](#) (const std::string &vardas)
- const std::string & [pavarde](#) () const
- void [setPavarde](#) (const std::string &pavarde)

Private Attributes

- GradeContainer [pazymiai_](#)
- int [egzaminas_](#) = 0
- double [vidurkis_](#) = 0.0
- double [mediana_](#) = 0.0

Additional Inherited Members

Protected Attributes inherited from Zmogus

- std::string [vardas_](#)
- std::string [pavarde_](#)

5.2.1 Detailed Description

```
template<typename GradeContainer>
class Studentas< GradeContainer >
```

Definition at line 60 of file [funkcijos.h](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Studentas() [1/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas () [inline]
Definition at line 62 of file funkcijos.h.
```

5.2.2.2 Studentas() [2/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas (
    const Studentas< GradeContainer > & other) [inline]
Definition at line 70 of file funkcijos.h.
```

5.2.2.3 Studentas() [3/3]

```
template<typename GradeContainer>
Studentas< GradeContainer >::Studentas (
    Studentas< GradeContainer > && other) [inline]
Definition at line 78 of file funkcijos.h.
```

5.2.2.4 ~Studentas()

```
template<typename GradeContainer>
Studentas< GradeContainer >::~~Studentas () [inline]
Definition at line 123 of file funkcijos.h.
```

5.2.3 Member Function Documentation

5.2.3.1 egzaminas()

```
template<typename GradeContainer>
int Studentas< GradeContainer >::egzaminas () const [inline]
```

Definition at line 135 of file [funkcijos.h](#).

5.2.3.2 mediana()

```
template<typename GradeContainer>
double Studentas< GradeContainer >::mediana () const [inline]
```

Definition at line 141 of file [funkcijos.h](#).

5.2.3.3 operator=() [1/2]

```
template<typename GradeContainer>
Studentas & Studentas< GradeContainer >::operator= (
    const Studentas< GradeContainer > & other) [inline]
```

Definition at line 93 of file [funkcijos.h](#).

5.2.3.4 operator=() [2/2]

```
template<typename GradeContainer>
Studentas & Studentas< GradeContainer >::operator= (
    Studentas< GradeContainer > && other) [inline]
```

Definition at line 105 of file [funkcijos.h](#).

5.2.3.5 pazymiai() [1/2]

```
template<typename GradeContainer>
GradeContainer & Studentas< GradeContainer >::pazymiai () [inline]
```

Definition at line 132 of file [funkcijos.h](#).

5.2.3.6 pazymiai() [2/2]

```
template<typename GradeContainer>
const GradeContainer & Studentas< GradeContainer >::pazymiai () const [inline]
```

Definition at line 133 of file [funkcijos.h](#).

5.2.3.7 setEgzaminas()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setEgzaminas (
    int egzaminas) [inline]
```

Definition at line 136 of file [funkcijos.h](#).

5.2.3.8 setMediana()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setMediana (
    double mediana) [inline]
```

Definition at line 142 of file [funkcijos.h](#).

5.2.3.9 setVidurkis()

```
template<typename GradeContainer>
void Studentas< GradeContainer >::setVidurkis (
    double vidurkis) [inline]
```

Definition at line 139 of file [funkcijos.h](#).

5.2.3.10 tipas()

```
template<typename GradeContainer>
std::string Studentas< GradeContainer >::tipas () const [inline], [override], [virtual]
Implements Zmogus.
Definition at line 144 of file funkcijos.h.
```

5.2.3.11 vidurkis()

```
template<typename GradeContainer>
double Studentas< GradeContainer >::vidurkis () const [inline]
Definition at line 138 of file funkcijos.h.
```

5.2.4 Member Data Documentation

5.2.4.1 egzaminas_

```
template<typename GradeContainer>
int Studentas< GradeContainer >::egzaminas_ = 0 [private]
Definition at line 148 of file funkcijos.h.
```

5.2.4.2 mediana_

```
template<typename GradeContainer>
double Studentas< GradeContainer >::mediana_ = 0.0 [private]
Definition at line 150 of file funkcijos.h.
```

5.2.4.3 pazymiai_

```
template<typename GradeContainer>
GradeContainer Studentas< GradeContainer >::pazymiai_ [private]
Definition at line 147 of file funkcijos.h.
```

5.2.4.4 vidurkis_

```
template<typename GradeContainer>
double Studentas< GradeContainer >::vidurkis_ = 0.0 [private]
Definition at line 149 of file funkcijos.h.
```

The documentation for this class was generated from the following file:

- funkcijos.h

5.3 TestStudent Class Reference

Public Member Functions

- [TestStudent](#) (const std::string &vardas, const std::string &pavarde, const [MyVector](#)< int > &pazymiai, int egzaminas, double vidurkis, double mediana)
- [TestStudent](#) (const [TestStudent](#) &other)
- [TestStudent](#) ([TestStudent](#) &&other)
- [TestStudent](#) & operator= (const [TestStudent](#) &other)
- [TestStudent](#) & operator= ([TestStudent](#) &&other)
- const std::string & vardas () const
- const std::string & pavarde () const
- [MyVector](#)< int > & pazymiai ()
- const [MyVector](#)< int > & pazymiai () const
- int egzaminas () const
- double vidurkis () const
- double mediana () const

- void [setVardas](#) (const std::string &vardas)
- void [setPavarde](#) (const std::string &pavarde)
- void [setEgzaminas](#) (int egzaminas)
- void [setVidurkis](#) (double vidurkis)
- void [setMediana](#) (double mediana)

Private Attributes

- std::string [vardas_](#)
- std::string [pavarde_](#)
- [MyVector](#)< int > [pazymiai_](#)
- int [egzaminas_](#) = 0
- double [vidurkis_](#) = 0.0
- double [mediana_](#) = 0.0

5.3.1 Detailed Description

Definition at line 4 of file [rule_of_five_test.cpp](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 TestStudent() [1/3]

```
TestStudent::TestStudent (
    const std::string & vardas,
    const std::string & pavarde,
    const MyVector< int > & pazymiai,
    int egzaminas,
    double vidurkis,
    double mediana) [inline]
```

Definition at line 6 of file [rule_of_five_test.cpp](#).

5.3.2.2 TestStudent() [2/3]

```
TestStudent::TestStudent (
    const TestStudent & other) [inline]
```

Definition at line 19 of file [rule_of_five_test.cpp](#).

5.3.2.3 TestStudent() [3/3]

```
TestStudent::TestStudent (
    TestStudent && other) [inline]
```

Definition at line 27 of file [rule_of_five_test.cpp](#).

5.3.3 Member Function Documentation

5.3.3.1 egzaminas()

```
int TestStudent::egzaminas () const [inline]
```

Definition at line 79 of file [rule_of_five_test.cpp](#).

5.3.3.2 mediana()

```
double TestStudent::mediana () const [inline]
```

Definition at line 81 of file [rule_of_five_test.cpp](#).

5.3.3.3 operator=() [1/2]

```
TestStudent & TestStudent::operator= (
    const TestStudent & other) [inline]
```

Definition at line 42 of file [rule_of_five_test.cpp](#).

5.3.3.4 operator=() [2/2]

```
TestStudent & TestStudent::operator= (
    TestStudent && other) [inline]
```

Definition at line 54 of file [rule_of_five_test.cpp](#).

5.3.3.5 pavarde()

```
const std::string & TestStudent::pavarde () const [inline]
```

Definition at line 76 of file [rule_of_five_test.cpp](#).

5.3.3.6 pazymiai() [1/2]

```
MyVector< int > & TestStudent::pazymiai () [inline]
```

Definition at line 77 of file [rule_of_five_test.cpp](#).

5.3.3.7 pazymiai() [2/2]

```
const MyVector< int > & TestStudent::pazymiai () const [inline]
```

Definition at line 78 of file [rule_of_five_test.cpp](#).

5.3.3.8 setEgzaminas()

```
void TestStudent::setEgzaminas (
    int egzaminas) [inline]
```

Definition at line 85 of file [rule_of_five_test.cpp](#).

5.3.3.9 setMediana()

```
void TestStudent::setMediana (
    double mediana) [inline]
```

Definition at line 87 of file [rule_of_five_test.cpp](#).

5.3.3.10 setPavarde()

```
void TestStudent::setPavarde (
    const std::string & pavarde) [inline]
```

Definition at line 84 of file [rule_of_five_test.cpp](#).

5.3.3.11 setVardas()

```
void TestStudent::setVardas (
    const std::string & vardas) [inline]
```

Definition at line 83 of file [rule_of_five_test.cpp](#).

5.3.3.12 setVidurkis()

```
void TestStudent::setVidurkis (
    double vidurkis) [inline]
```

Definition at line 86 of file [rule_of_five_test.cpp](#).

5.3.3.13 vardas()

```
const std::string & TestStudent::vardas () const [inline]
```

Definition at line 75 of file [rule_of_five_test.cpp](#).

5.3.3.14 vidurkis()

double TestStudent::vidurkis () const [inline]
Definition at line 80 of file [rule_of_five_test.cpp](#).

5.3.4 Member Data Documentation

5.3.4.1 egzaminas_

int TestStudent::egzaminas_ = 0 [private]
Definition at line 93 of file [rule_of_five_test.cpp](#).

5.3.4.2 mediana_

double TestStudent::mediana_ = 0.0 [private]
Definition at line 95 of file [rule_of_five_test.cpp](#).

5.3.4.3 pavarde_

std::string TestStudent::pavarde_ [private]
Definition at line 91 of file [rule_of_five_test.cpp](#).

5.3.4.4 pazymiai_

[MyVector](#)<int> TestStudent::pazymiai_ [private]
Definition at line 92 of file [rule_of_five_test.cpp](#).

5.3.4.5 vardas_

std::string TestStudent::vardas_ [private]
Definition at line 90 of file [rule_of_five_test.cpp](#).

5.3.4.6 vidurkis_

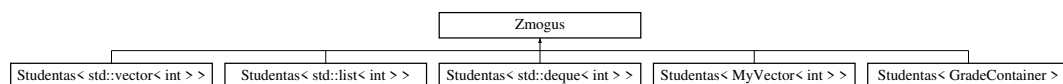
double TestStudent::vidurkis_ = 0.0 [private]
Definition at line 94 of file [rule_of_five_test.cpp](#).

The documentation for this class was generated from the following file:

- [rule_of_five_test.cpp](#)

5.4 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- [Zmogus](#) (const [Zmogus](#) &other)
- [Zmogus](#) ([Zmogus](#) &&other)
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &other)
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&other)
- const std::string & [vardas](#) () const
- void [setVardas](#) (const std::string &vardas)
- const std::string & [pavarde](#) () const
- void [setPavarde](#) (const std::string &pavarde)
- virtual std::string [tipas](#) () const =0

Protected Attributes

- std::string [vardas_](#)
- std::string [pavarde_](#)

5.4.1 Detailed Description

Definition at line 8 of file [funkcijos.h](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde) [inline]
```

Definition at line 10 of file [funkcijos.h](#).

5.4.2.2 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & other) [inline]
```

Definition at line 14 of file [funkcijos.h](#).

5.4.2.3 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && other) [inline]
```

Definition at line 18 of file [funkcijos.h](#).

5.4.2.4 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Definition at line 41 of file [funkcijos.h](#).

5.4.3 Member Function Documentation

5.4.3.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other) [inline]
```

Definition at line 25 of file [funkcijos.h](#).

5.4.3.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other) [inline]
```

Definition at line 33 of file [funkcijos.h](#).

5.4.3.3 pavarde()

```
const std::string & Zmogus::pavarde () const [inline]
```

Definition at line 49 of file [funkcijos.h](#).

5.4.3.4 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & pavarde) [inline]
```

Definition at line 50 of file [funkcijos.h](#).

5.4.3.5 setVardas()

```
void Zmogus::setVardas (  
    const std::string & vardas) [inline]
```

Definition at line 47 of file [funkcijos.h](#).

5.4.3.6 vardas()

```
const std::string & Zmogus::vardas () const [inline]
```

Definition at line 46 of file [funkcijos.h](#).

5.4.4 Member Data Documentation

5.4.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Definition at line 56 of file [funkcijos.h](#).

5.4.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Definition at line 55 of file [funkcijos.h](#).

The documentation for this class was generated from the following file:

- [funkcijos.h](#)

Chapter 6

File Documentation

6.1 funkcijos.h

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "library.h"
00005 #include "patikrinimai.h"
00006 #include "vector.h"
00007
00008 class Zmogus {
00009 public:
00010     Zmogus(const std::string& vardas, const std::string& pavarde)
00011         : vardas_(vardas),
00012           pavarde_(pavarde) {}
00013
00014     Zmogus(const Zmogus& other)
00015         : vardas_(other.vardas_),
00016           pavarde_(other.pavarde_) {}
00017
00018     Zmogus(Zmogus&& other)
00019         : vardas_(std::move(other.vardas_)),
00020           pavarde_(std::move(other.pavarde_)) {
00021         other.vardas_.clear();
00022         other.pavarde_.clear();
00023     }
00024
00025     Zmogus& operator=(const Zmogus& other) {
00026         if (this != &other) {
00027             vardas_ = other.vardas_;
00028             pavarde_ = other.pavarde_;
00029         }
00030         return *this;
00031     }
00032
00033     Zmogus& operator=(Zmogus&& other) {
00034         if (this != &other) {
00035             vardas_ = std::move(other.vardas_);
00036             pavarde_ = std::move(other.pavarde_);
00037         }
00038         return *this;
00039     }
00040
00041     virtual ~Zmogus() {
00042         vardas_.clear();
00043         pavarde_.clear();
00044     }
00045
00046     const std::string& vardas() const { return vardas_; }
00047     void setVardas(const std::string& vardas) { vardas_ = vardas; }
00048
00049     const std::string& pavarde() const { return pavarde_; }
00050     void setPavarde(const std::string& pavarde) { pavarde_ = pavarde; }
00051
00052     virtual std::string tipas() const = 0;
00053
00054 protected:
00055     std::string vardas_;
00056     std::string pavarde_;
00057 };
00058
00059 template <typename GradeContainer>
00060 class Studentas : public Zmogus {
00061 public:
```

```

00062 Studentas()
00063 : Zmogus("", ""),
00064   pazymiai_(),
00065   egzaminas_(0),
00066   vidurkis_(0.0),
00067   mediana_(0.0) {}
00068
00069 // Copy constructor.
00070 Studentas(const Studentas& other)
00071 : Zmogus(other),
00072   pazymiai_(other.pazymiai_),
00073   egzaminas_(other egzaminas_),
00074   vidurkis_(other.vidurkis_),
00075   mediana_(other.mediana_) {}
00076
00077 // Move constructor.
00078 Studentas(Studentas&& other)
00079 : Zmogus(std::move(other)),
00080   pazymiai_(std::move(other.pazymiai_)),
00081   egzaminas_(other egzaminas_),
00082   vidurkis_(other.vidurkis_),
00083   mediana_(other.mediana_) {
00084   other.vardas_.clear();
00085   other.pavarde_.clear();
00086   other.pazymiai_.clear();
00087   other egzaminas_ = 0;
00088   other.vidurkis_ = 0.0;
00089   other.mediana_ = 0.0;
00090 }
00091
00092 // Copy assignment operator.
00093 Studentas& operator=(const Studentas& other) {
00094     if (this != &other) {
00095         Zmogus::operator=(other);
00096         pazymiai_ = other.pazymiai_;
00097         egzaminas_ = other egzaminas_;
00098         vidurkis_ = other.vidurkis_;
00099         mediana_ = other.mediana_;
00100     }
00101     return *this;
00102 }
00103
00104 // Move assignment operator.
00105 Studentas& operator=(Studentas&& other) {
00106     if (this != &other) {
00107         Zmogus::operator=(std::move(other));
00108         pazymiai_ = std::move(other.pazymiai_);
00109         egzaminas_ = other egzaminas_;
00110         vidurkis_ = other.vidurkis_;
00111         mediana_ = other.mediana_;
00112         other.vardas_.clear();
00113         other.pavarde_.clear();
00114         other.pazymiai_.clear();
00115         other egzaminas_ = 0;
00116         other.vidurkis_ = 0.0;
00117         other.mediana_ = 0.0;
00118     }
00119     return *this;
00120 }
00121
00122 // Destructor.
00123 ~Studentas() {
00124     vardas_.clear();
00125     pavarde_.clear();
00126     pazymiai_.clear();
00127     egzaminas_ = 0;
00128     vidurkis_ = 0.0;
00129     mediana_ = 0.0;
00130 }
00131
00132 GradeContainer& pazymiai() { return pazymiai_; }
00133 const GradeContainer& pazymiai() const { return pazymiai_; }
00134
00135 int egzaminas() const { return egzaminas_; }
00136 void setEgzaminas(int egzaminas) { egzaminas_ = egzaminas; }
00137
00138 double vidurkis() const { return vidurkis_; }
00139 void setVidurkis(double vidurkis) { vidurkis_ = vidurkis; }
00140
00141 double mediana() const { return mediana_; }
00142 void setMediana(double mediana) { mediana_ = mediana; }
00143
00144 std::string tipas() const override { return "Studentas"; }
00145
00146 private:
00147     GradeContainer pazymiai_;
00148     int egzaminas_ = 0;

```

```

00149     double vidurkis_ = 0.0;
00150     double mediana_ = 0.0;
00151 };
00152 using VectorStudent = Studentas<std::vector<int>>;
00153 using ListStudent = Studentas<std::list<int>>;
00154 using DequeStudent = Studentas<std::deque<int>>;
00155 using MyVectorStudent = Studentas<MyVector<int>>;
00156
00157 using VectorContainer = std::vector<VectorStudent>;
00158 using ListContainer = std::list<ListStudent>;
00159 using DequeContainer = std::deque<DequeStudent>;
00160 using MyVectorContainer = MyVector<MyVectorStudent>;
00161
00162 template <typename GradeContainer>
00163 std::ostream& operator<(std::ostream& out, const Studentas<GradeContainer>& studentas) {
00164     out << studentas.vardas() << " "
00165         << studentas.pavarde() << " "
00166         << studentas.egzaminas() << " "
00167         << studentas.pazymiai().size();
00168
00169     for (int pazymis : studentas.pazymiai()) {
00170         out << " " << pazymis;
00171     }
00172
00173     return out;
00174 }
00175
00176 template <typename GradeContainer>
00177 std::istream& operator<(std::istream& in, Studentas<GradeContainer>& studentas) {
00178     std::string vardas;
00179     std::string pavarde;
00180     int egzaminas = 0;
00181     int pazymiu_kiekis = 0;
00182
00183     if (!(in >> vardas >> pavarde >> egzaminas >> pazymiu_kiekis)) {
00184         return in;
00185     }
00186
00187     studentas.setVardas(vardas);
00188     studentas.setPavarde(pavarde);
00189     studentas.setEgzaminas(egzaminas);
00190     studentas.pazymiai().clear();
00191
00192     for (int i = 0; i < pazymiu_kiekis; ++i) {
00193         int pazymis = 0;
00194         if (!(in >> pazymis)) {
00195             return in;
00196         }
00197         studentas.pazymiai().push_back(pazymis);
00198     }
00199
00200     return in;
00201 }
00202
00203
00204 template <typename T, typename Allocator, typename Compare>
00205 void sort_container(std::list<T, Allocator>& container, Compare comp) {
00206     container.sort(comp);
00207 }
00208
00209 template <typename Container, typename Compare>
00210 void sort_container(Container& container, Compare comp) {
00211     std::sort(container.begin(), container.end(), comp);
00212 }
00213
00214 template <typename GradeContainer>
00215 double skaiciuoti_mediana(GradeContainer& paz) {
00216     if (paz.empty()) {
00217         return 0.0;
00218     }
00219
00220     if constexpr (std::is_same_v<GradeContainer, std::list<int>>) {
00221         paz.sort(std::less<int>{});
00222     } else {
00223         std::sort(paz.begin(), paz.end(), std::less<int>{});
00224     }
00225
00226     const std::size_t dydis = paz.size();
00227     auto mid = paz.begin();
00228     std::advance(mid, static_cast<long>(dydis / 2));
00229     if (dydis % 2 == 0) {
00230         auto left = mid;
00231         --left;
00232         return (*left + *mid) / 2.0;
00233     }
00234     return static_cast<double>(*mid);
00235 }

```

```

00236
00237 template <typename Student>
00238 void paz_sk(Student& temp, int& m) {
00239     std::string input;
00240     while (true) {
00241         std::cout << "Kiek pazymiu turi " << temp.vardas() << " " << temp.pavarde() << "? ";
00242         if (!read_input(input)) {
00243             return;
00244         }
00245         m = validation(input);
00246         if (m == 0) {
00247             std::cout << "Ne skaicius!" << std::endl;
00248         } else if (m < 1) {
00249             std::cout << "Iveskite tinkama sk." << std::endl;
00250         } else {
00251             break;
00252         }
00253     }
00254 }
00255
00256 template <typename Student>
00257 void paz_ivestis_ranka(Student& temp, int m) {
00258     std::string input;
00259     for (int j = 0; j < m; ++j) {
00260         std::cout << "Iveskite " << j + 1 << " pazymi: ";
00261         if (!read_input(input)) {
00262             return;
00263         }
00264         const int pazymis = validation(input);
00265         if (pazymis > 10 || pazymis < 1) {
00266             std::cout << "Netinkamas sk. Bandykite dar karta" << std::endl;
00267             --j;
00268             continue;
00269         }
00270         temp.pazymiai().push_back(pazymis);
00271     }
00272 }
00273
00274 template <typename Student>
00275 void egz_ivestis_ranka(Student& temp) {
00276     std::string input;
00277     while (true) {
00278         std::cout << "Koks yra " << temp.vardas() << " " << temp.pavarde() << " egzamino rezultatas? ";
00279         if (!read_input(input)) {
00280             return;
00281         }
00282         const int egz = validation(input);
00283         if (egz > 10 || egz < 1) {
00284             std::cout << "Netinkamas sk. Bandykite dar karta" << std::endl;
00285             continue;
00286         }
00287         temp.setEgzaminas(egz);
00288         break;
00289     }
00290 }
00291
00292 template <typename Student>
00293 void vardu_ivedimas_ranka(Student& temp, std::size_t current_count) {
00294     while (true) {
00295         std::string vardas;
00296         std::string pavarde;
00297         std::cout << "Koks yra " << current_count + 1 << " studento vardas ir pavarde? ";
00298         if (!(std::cin >> vardas >> pavarde)) {
00299             if (std::cin.eof()) {
00300                 std::cout << "Ivestis nutraukta." << std::endl;
00301                 return;
00302             }
00303             std::cin.clear();
00304             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00305             std::cout << "Netinkama ivestis. Bandykite dar karta." << std::endl;
00306             continue;
00307         }
00308         if (valid_name(vardas) && valid_name(pavarde)) {
00309             temp.setVardas(vardas);
00310             temp.setPavarde(pavarde);
00311             break;
00312         }
00313         std::cout << "Iveskite tinkama varda ir pavarde" << std::endl;
00314     }
00315 }
00316
00317 template <typename Student>
00318 void paz_ivestis_random(Student& temp, int m) {
00319     std::random_device rd;
00320     std::mt19937 gen(rd());
00321     std::uniform_int_distribution<> dist(1, 10);
00322     for (int i = 0; i < m; ++i) {

```

```

00323         temp.pazymiai().push_back(dist(gen));
00324     }
00325 }
00326
00327 template <typename Student>
00328 void egz_ivestis_random(Student& temp) {
00329     std::random_device rd;
00330     std::mt19937 gen(rd());
00331     std::uniform_int_distribution<> dist(1, 10);
00332     temp.setEgzaminas(dist(gen));
00333 }
00334
00335 template <typename Student>
00336 void vardu_ivedimas_random(Student& temp) {
00337     static const std::vector<std::string> vard = {
00338         "Artur", "Simas", "Romas", "Patrikas", "Rokas",
00339         "Ignas", "Tomas", "Rugile", "Aiste", "Martynas"
00340     };
00341     static const std::vector<std::string> pav = {
00342         "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
00343         "Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"
00344     };
00345     std::random_device rd;
00346     std::mt19937 gen(rd());
00347     std::uniform_int_distribution<> dist(0, 9);
00348     const int r = dist(gen);
00349     temp.setVardas(vard[r]);
00350     temp.setPavarde(pav[r]);
00351 }
00352 }
00353
00354 template <typename StudentContainer>
00355 void vidurkis(StudentContainer& stud) {
00356     for (auto& studentas : stud) {
00357         double sum = 0.0;
00358         for (int pazymis : studentas.pazymiai()) {
00359             sum += pazymis;
00360         }
00361         studentas.setVidurkis((sum / static_cast<double>(studentas.pazymiai().size())) * 0.4
00362             + studentas.egzaminas() * 0.6);
00363     }
00364 }
00365
00366 template <typename StudentContainer>
00367 void mediana(StudentContainer& stud) {
00368     for (auto& studentas : stud) {
00369         studentas.setMediana(skaiciuoti_mediana(studentas.pazymiai()));
00370     }
00371 }
00372
00373 template <typename StudentContainer>
00374 void isvestis(StudentContainer& stud) {
00375     std::string input;
00376     while (true) {
00377         std::cout << "Ka noretumet pamatyti? Mediana - 1, arba Vidurki - 2 ";
00378         if (!read_input(input)) {
00379             return;
00380         }
00381
00382         const int choice = validation(input);
00383         if (choice != 1 && choice != 2) {
00384             std::cout << "Iveskite tinkama sk!" << std::endl;
00385             continue;
00386         }
00387
00388         if (choice == 1) {
00389             std::cout << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00390                 << std::setw(20) << "Galutinis (Med.)" << std::endl;
00391         } else {
00392             std::cout << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00393                 << std::setw(20) << "Galutinis (Vid.)" << std::endl;
00394         }
00395         std::cout <<
00396             "-----" << std::endl;
00397         for (const auto& studentas : stud) {
00398             std::cout << std::fixed << std::left << std::setw(15) << studentas.vardas()
00399                 << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)
00400                 << (choice == 1 ? studentas.mediana() : studentas.vidurkis()) << std::endl;
00401         }
00402     }
00403 }
00404
00405 template <typename StudentContainer>
00406 void rusiavimas(StudentContainer& stud, double& laikas) {
00407     std::string input;
00408     while (true) {

```

```

00409         std::cout << "Rusiukite studentus pagal: 1 - vardas, 2 - pavarde, 3 - vidurki, 4 - mediana ";
00410         if (!read_input(input)) {
00411             return;
00412         }
00413
00414         const auto pradzia = std::chrono::high_resolution_clock::now();
00415         const int choice = validation(input);
00416         if (choice == 1) {
00417             sort_container(stud, [](const auto& a, const auto& b) { return a.vardas() < b.vardas();
00418         });
00419         } else if (choice == 2) {
00420             sort_container(stud, [](const auto& a, const auto& b) { return a.pavarde() < b.pavarde();
00421         });
00422         } else if (choice == 3) {
00423             sort_container(stud, [](const auto& a, const auto& b) { return a.vidurkis() <
00424         b.vidurkis(); });
00425         } else if (choice == 4) {
00426             sort_container(stud, [](const auto& a, const auto& b) { return a.mediana() < b.mediana();
00427         });
00428         } else {
00429             std::cout << "Iveskite tinkama sk!" << std::endl;
00430             continue;
00431         }
00432         const auto pabaiga = std::chrono::high_resolution_clock::now();
00433         const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00434         laikas += trukme.count();
00435         return;
00436     }
00437 }
00438
00439 inline void failu_kurimas(const std::string& name, int zmones, int m, double& laikas) {
00440     std::random_device rd;
00441     std::mt19937 gen(rd());
00442     std::uniform_int_distribution<> dist(1, 10);
00443
00444     const auto pradzia = std::chrono::high_resolution_clock::now();
00445
00446     std::ofstream failas(name);
00447     failas << "vardas pavarde ";
00448     for (int i = 0; i < m; ++i) {
00449         failas << " ND" << i + 1;
00450     }
00451     failas << " egz" << std::endl;
00452     for (int i = 0; i < zmones; ++i) {
00453         failas << "vardas" << i + 1 << " pavarde" << i + 1;
00454         for (int j = 0; j < m; ++j) {
00455             failas << " " << dist(gen);
00456         }
00457         failas << " " << dist(gen) << std::endl;
00458     }
00459
00460     const auto pabaiga = std::chrono::high_resolution_clock::now();
00461     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00462     std::cout << zmones << " Rasymas uztruko " << trukme.count() << " ms" << std::endl;
00463     laikas += trukme.count();
00464 }
00465
00466 template <typename StudentContainer>
00467 void isvestis_failas(StudentContainer& stud) {
00468     std::string input;
00469     while (true) {
00470         std::cout << "Duomenis rasyti: 1 - i konsole, 2 - i atskira faila: ";
00471         if (!read_input(input)) {
00472             return;
00473         }
00474
00475         const int choice = validation(input);
00476         if (choice != 1 && choice != 2) {
00477             std::cout << "Iveskite tinkama sk!" << std::endl;
00478             continue;
00479         }
00480
00481         std::ostream* out = &std::cout;
00482         std::ofstream rezfailas;
00483         if (choice == 2) {
00484             rezfailas.open("rezultatai.txt");
00485             out = &rezfailas;
00486         }
00487
00488         *out << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00489             << std::setw(20) << "Galutinis (Vid.)" << std::setw(10) << "Galutinis (Med.)" << std::endl;
00490         *out << "-----" << std::endl;
00491         for (const auto& studentas : stud) {
00492             *out << std::fixed << std::left << std::setw(15) << studentas.vardas()
00493                 << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)

```



```

00491         « studentas.vidurkis() « std::setw(9) « studentas.mediana() « std::endl;
00492     }
00493     return;
00494 }
00495 }
00496
00497 template <typename StudentContainer>
00498 void skaitymas(StudentContainer& stud, const std::string& input, double& laikas) {
00499     using Student = typename StudentContainer::value_type;
00500
00501     std::string line;
00502     int pazymis = 0;
00503     std::size_t capacity_hits = 0;
00504
00505     try {
00506         const std::filesystem::path failo_kelias = input;
00507         if (!std::filesystem::exists(failo_kelias)) {
00508             throw std::runtime_error("Toks failas nurodytame aplanke nerastas!");
00509         }
00510         if (!std::filesystem::is_regular_file(failo_kelias)) {
00511             throw std::runtime_error("Nurodytas kelias nera failas!");
00512         }
00513
00514         const auto pradzia = std::chrono::high_resolution_clock::now();
00515         std::ifstream duomfailas(input);
00516         if (!duomfailas.is_open()) {
00517             throw std::runtime_error("Nepavyko atidaryti failo!");
00518         }
00519         if (duomfailas.peek() == std::ifstream::traits_type::eof()) {
00520             throw std::runtime_error("Failas tuscias!");
00521         }
00522
00523         std::stringstream buffer;
00524         buffer « duomfailas.rdbuf();
00525
00526         std::getline(buffer, line);
00527         while (std::getline(buffer, line)) {
00528             if (line.empty()) {
00529                 continue;
00530             }
00531
00532             Student temp;
00533             std::istringstream laik(line);
00534             std::string vardas;
00535             std::string pavarde;
00536             if (!(laik » vardas » pavarde)) {
00537                 throw std::runtime_error("Netinkamas failo formatas.");
00538             }
00539             temp.setVardas(vardas);
00540             temp.setPavarde(pavarde);
00541
00542             while (laik » pazymis) {
00543                 if (pazymis < 1 || pazymis > 10) {
00544                     throw std::runtime_error("Pazymiai faile turi buti nuo 1 iki 10.");
00545                 }
00546                 temp.pazymiai().push_back(pazymis);
00547             }
00548
00549             if (laik.fail() && !laik.eof()) {
00550                 throw std::runtime_error("Netinkamas pazymio formatas faile.");
00551             }
00552             if (temp.pazymiai().empty()) {
00553                 throw std::runtime_error("Studentas neturi nei vieno pazymio.");
00554             }
00555
00556             temp.setEgzaminas(temp.pazymiai().back());
00557             temp.pazymiai().pop_back();
00558             if (temp.pazymiai().empty()) {
00559                 throw std::runtime_error("Truksta namu darbu pazymiu.");
00560             }
00561             if constexpr (std::is_same_v<StudentContainer, VectorContainer>
00562                 || std::is_same_v<StudentContainer, MyVectorContainer>) {
00563                 if (stud.size() == stud.capacity()) {
00564                     ++capacity_hits;
00565                 }
00566             }
00567             stud.push_back(temp);
00568         }
00569
00570         const auto pabaiga = std::chrono::high_resolution_clock::now();
00571         const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00572         std::cout « "Skaitymas uztruko " « trukme.count() « " ms" « std::endl;
00573         laikas += trukme.count();
00574         if constexpr (std::is_same_v<StudentContainer, VectorContainer>
00575             || std::is_same_v<StudentContainer, MyVectorContainer>) {
00576             std::cout « "Size buvo lygu capacity " « capacity_hits « " kartu" « std::endl;
00577         }

```

```

00578     } catch (const std::filesystem::filesystem_error& e) {
00579         std::cout << "Failu sistemos klaida: " << e.what() << std::endl;
00580     } catch (const std::exception& e) {
00581         std::cout << e.what() << std::endl;
00582     }
00583 }
00584
00585 template <typename StudentContainer>
00586 void skirstymas(StudentContainer& stud, StudentContainer& maladiiec, StudentContainer& lopai, double&
    laikas) {
00587     const auto pradzia = std::chrono::high_resolution_clock::now();
00588     (void)stud;
00589
00590     auto border = std::partition(maladiiec.begin(), maladiiec.end(), [](const auto& studentas) {
00591         return studentas.vidurkis() >= 5;
00592     });
00593
00594     for (auto it = border; it != maladiiec.end(); ++it) {
00595         lopai.push_back(*it);
00596     }
00597     maladiiec.erase(border, maladiiec.end());
00598
00599     const auto pabaiga = std::chrono::high_resolution_clock::now();
00600     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00601     std::cout << "Skirstymas i 2 konteinerius truko " << trukme.count() << " ms" << std::endl;
00602     laikas += trukme.count();
00603 }
00604
00605 template <typename StudentContainer>
00606 void rasymas(const StudentContainer& a, const std::string& name, double& laikas) {
00607     const auto pradzia = std::chrono::high_resolution_clock::now();
00608
00609     std::ofstream rezfailas(name);
00610     rezfailas << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde"
00611         << std::setw(20) << "Galutinis (Vid.)" << std::setw(10) << "Galutinis (Med.)" << std::endl;
00612     rezfailas << "-----"
00613 << std::endl;
00614     for (const auto& studentas : a) {
00615         rezfailas << std::fixed << std::left << std::setw(15) << studentas.vardas()
00616             << std::setw(15) << studentas.pavarde() << std::setw(9) << std::setprecision(2)
00617             << studentas.vidurkis() << std::setw(9) << studentas.mediana() << std::endl;
00618     }
00619
00620     const auto pabaiga = std::chrono::high_resolution_clock::now();
00621     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00622     std::cout << name << " Rasymas truko " << trukme.count() << " ms" << std::endl;
00623     laikas += trukme.count();
00624 }
00625
00626 template <typename StudentContainer>
00627 void testavimas(StudentContainer& stud, StudentContainer& maladiiec, StudentContainer& lopai, double&
    laikas) {
00628     vidurkis(maladiiec);
00629     mediana(maladiiec);
00630     sort_ascending(maladiiec, laikas);
00631     skirstymas(stud, maladiiec, lopai, laikas);
00632     rasymas(maladiiec, "maladiiec.txt", laikas);
00633     rasymas(lopai, "lopai.txt", laikas);
00634     std::cout << "Darbas su failu uztruko " << laikas << " ms" << std::endl;
00635     std::cout << std::endl;
00636 }
00637
00638 template <typename StudentContainer>
00639 void tyrimai_5(StudentContainer& stud, StudentContainer& maladiiec, StudentContainer& lopai, double&
    laikas) {
00640     while (true) {
00641         std::string ivestis;
00642         std::cout << "Kuri faila norite skaityti? 1 - studentai1000.txt, 2 - studentai10000.txt, 3 -
            studentai100000.txt, 4 - studentai1000000.txt, 5 - studentai10000000.txt, 6 - baigti ";
00643         if (!read_input(ivistis)) {
00644             return;
00645         }
00646
00647         const int p = validation(ivistis);
00648         if (p == 6) {
00649             break;
00650         }
00651
00652         std::string failo_vardas;
00653         if (p == 1) {
00654             failo_vardas = "studentai1000.txt";
00655         } else if (p == 2) {
00656             failo_vardas = "studentai10000.txt";
00657         } else if (p == 3) {
00658             failo_vardas = "studentai100000.txt";
00659         } else if (p == 4) {
00660             failo_vardas = "studentai1000000.txt";
00661         }
    }

```

```

00660         } else if (p == 5) {
00661             failo_vardas = "studentai10000000.txt";
00662         } else {
00663             std::cout << "Iveskite tinkama sk. " << std::endl;
00664             continue;
00665         }
00666         stud.clear();
00667         maladiec.clear();
00668         lopai.clear();
00669         laikas = 0;
00670
00671         skaitymas(maladiec, failo_vardas, laikas);
00672         testavimas(stud, maladiec, lopai, laikas);
00673     }
00674 }
00675 }
00676
00677 template <typename StudentContainer>
00678 int run_program(const std::string& konteinerio_pavadinimas) {
00679     using Student = typename StudentContainer::value_type;
00680
00681     StudentContainer stud;
00682     StudentContainer maladiec;
00683     StudentContainer lopai;
00684     std::string input;
00685     double b = 0.0;
00686     double laikas = 0.0;
00687
00688     std::cout << "Naudojamas konteineris: " << konteinerio_pavadinimas << std::endl;
00689     std::cout << "Studentu Vardu ir pazymiu ivedimu sistema, skirta medianos bei vidurkio
apskaiciavimui" << std::endl;
00690     while (true) {
00691         Student temp;
00692         int m = 0;
00693         while (true) {
00694             std::cout << "1 - ranka, 2 - generuoti tik pazymius, 3 - generuoti studentu vardus,
pavardes ir pazymius, 4 - skaityti duomenis is failo, 5 - generuoti failus, 6 - baigti darba: ";
00695             if (!read_input(input)) {
00696                 return 0;
00697             }
00698
00699             const int choice = validation(input);
00700             if (choice == 1) {
00701                 vardu_ivedimas_ranka(temp, stud.size());
00702                 if (temp.vardas().empty() || temp.pavarde().empty()) {
00703                     break;
00704                 }
00705                 paz_sk(temp, m);
00706                 if (m < 1) {
00707                     break;
00708                 }
00709                 paz_ivestis_ranka(temp, m);
00710                 if (temp.pazymiai().size() != static_cast<std::size_t>(m)) {
00711                     break;
00712                 }
00713                 egz_ivestis_ranka(temp);
00714                 if (temp.egzaminas() == 0) {
00715                     break;
00716                 }
00717                 stud.push_back(temp);
00718             } else if (choice == 2) {
00719                 vardu_ivedimas_ranka(temp, stud.size());
00720                 if (temp.vardas().empty() || temp.pavarde().empty()) {
00721                     break;
00722                 }
00723                 paz_sk(temp, m);
00724                 if (m < 1) {
00725                     break;
00726                 }
00727                 paz_ivestis_random(temp, m);
00728                 egz_ivestis_random(temp);
00729                 stud.push_back(temp);
00730             } else if (choice == 3) {
00731                 vardu_ivedimas_random(temp);
00732                 paz_sk(temp, m);
00733                 if (m < 1) {
00734                     break;
00735                 }
00736                 paz_ivestis_random(temp, m);
00737                 egz_ivestis_random(temp);
00738                 stud.push_back(temp);
00739             } else if (choice == 4) {
00740                 tyrimai_5(stud, maladiec, lopai, laikas);
00741                 break;
00742             } else if (choice == 5) {
00743                 m = paz_sk();
00744                 failu_kurimas("studentai1000.txt", 1000, m, b);

```

```

00745         failu_kurimas("studentai10000.txt", 10000, m, b);
00746         failu_kurimas("studentai100000.txt", 100000, m, b);
00747         failu_kurimas("studentai1000000.txt", 1000000, m, b);
00748         failu_kurimas("studentai10000000.txt", 10000000, m, b);
00749         std::cout << std::endl;
00750         tyrimai_5(stud, maladiet, lopai, laikas);
00751         break;
00752     } else if (choice == 6) {
00753         std::cout << "Sekmingai baigete studentu duomeni ivedima!" << std::endl;
00754         break;
00755     } else {
00756         std::cout << "Iveskite tinkama sk!" << std::endl;
00757     }
00758 }
00759 break;
00760 }
00761
00762 if (stud.empty()) {
00763     return 0;
00764 }
00765
00766 vidurkis(stud);
00767 mediana(stud);
00768 if (stud.size() > 1) {
00769     rusiavimas(stud, b);
00770 }
00771 isvestis_failas(stud);
00772 return 0;
00773 }
00774 template <typename Container>
00775 void sort_ascending(Container& c, double& laikas) {
00776     const auto pradzia = std::chrono::high_resolution_clock::now();
00777     if constexpr (std::is_same_v<Container, std::list<typename Container::value_type> > {
00778         c.sort([](const auto& a, const auto& b) {
00779             return a.vidurkis() < b.vidurkis();
00780         });
00781     } else {
00782         std::sort(c.begin(), c.end(), [](const auto& a, const auto& b) {
00783             return a.vidurkis() < b.vidurkis();
00784         });
00785     }
00786     const auto pabaiga = std::chrono::high_resolution_clock::now();
00787     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00788     std::cout << "Sort truko " << trukme.count() << std::endl;
00789     laikas += trukme.count();
00790 }
00791
00792 #endif

```

6.2 library.h

```

00001 #ifndef LIBRARY_H
00002 #define LIBRARY_H
00003
00004 #include <algorithm>
00005 #include <chrono>
00006 #include <cctype>
00007 #include <cstdint>
00008 #include <cassert>
00009 #include <deque>
00010 #include <filesystem>
00011 #include <fstream>
00012 #include <iomanip>
00013 #include <iostream>
00014 #include <iterator>
00015 #include <limits>
00016 #include <list>
00017 #include <random>
00018 #include <sstream>
00019 #include <stdexcept>
00020 #include <string>
00021 #include <type_traits>
00022 #include <utility>
00023 #include <vector>
00024
00025 #endif

```

6.3 main.cpp

```

00001 #include "funkcijos.h"
00002 #include "library.h"

```

```

00003 #include "patikrinimai.h"
00004
00005 int main() {
00006     std::string input;
00007
00008     while (true) {
00009         std::cout << "Pasirinkite konteineri: 1 - vector, 2 - list, 3 - deque, 4 - mano vektorius: ";
00010         if (!read_input(input)) {
00011             return 0;
00012         }
00013
00014         const int choice = validation(input);
00015         if (choice == 1) {
00016             return run_program<VectorContainer>("vector");
00017         }
00018         if (choice == 2) {
00019             return run_program<ListContainer>("list");
00020         }
00021         if (choice == 3) {
00022             return run_program<DequeContainer>("deque");
00023         }
00024         if (choice == 4) {
00025             return run_program<MyVectorContainer>("MyVector");
00026         }
00027
00028         std::cout << "Iveskite tinkama sk!" << std::endl;
00029     }
00030 }

```

6.4 objektinis_patikrinimai.cpp

```

00001
00002 #include "library.h"
00003 #include "patikrinimai.h"
00004
00005 bool valid_name(const std::string& s) {
00006     if (s.empty()) {
00007         return false;
00008     }
00009
00010     for (unsigned char c : s) {
00011         if (!std::isalpha(c) && c != '-') {
00012             return false;
00013         }
00014     }
00015     return true;
00016 }
00017
00018 int validation(const std::string& a) {
00019     try {
00020         return std::stoi(a);
00021     } catch (const std::exception&) {
00022         return 0;
00023     }
00024 }
00025
00026 bool read_input(std::string& input) {
00027     try {
00028         if (std::cin >> input) {
00029             return true;
00030         }
00031
00032         if (std::cin.eof()) {
00033             std::cout << "Ivestis nutraukta." << std::endl;
00034             return false;
00035         }
00036
00037         std::cin.clear();
00038         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00039         std::cout << "Netinkama ivestis. Bandykite dar karta." << std::endl;
00040     } catch (const std::exception&) {
00041         std::cin.clear();
00042         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00043         std::cout << "Klaida skaitant ivesti. Bandykite dar karta." << std::endl;
00044     }
00045     return false;
00046 }
00047
00048 }
00049
00050 int paz_sk() {
00051     std::string input;
00052     int pazsk = 0;
00053     while (true) {

```

```

00054         std::cout << "Kiek pazymiu tures studentai sarase? ";
00055         std::cin >> input;
00056         pazsk = validation(input);
00057         if (pazsk > 0) {
00058             return pazsk;
00059         }
00060         std::cout << "iveskite tinkama sk! " << std::endl;
00061     }
00062 }

```

6.5 patikrinimai.h

```

00001 #ifndef PATIKRINIMAI_H
00002 #define PATIKRINIMAI_H
00003
00004 #include "library.h"
00005
00006 bool valid_name(const std::string& s);
00007 int validation(const std::string& a);
00008 bool read_input(std::string& input);
00009 int paz_sk();
00010
00011 #endif

```

6.6 rule_of_five_test.cpp

```

00001 #include "vector.h"
00002 #include "library.h"
00003
00004 class TestStudent {
00005 public:
00006     TestStudent(const std::string& vardas,
00007                 const std::string& pavarde,
00008                 const MyVector<int>& pazymiai,
00009                 int egzaminas,
00010                 double vidurkis,
00011                 double mediana)
00012         : vardas_(vardas),
00013           pavarde_(pavarde),
00014           pazymiai_(pazymiai),
00015           egzaminas_(egzaminas),
00016           vidurkis_(vidurkis),
00017           mediana_(mediana) {}
00018
00019     TestStudent(const TestStudent& other)
00020         : vardas_(other.vardas_),
00021           pavarde_(other.pavarde_),
00022           pazymiai_(other.pazymiai_),
00023           egzaminas_(other.egzaminas_),
00024           vidurkis_(other.vidurkis_),
00025           mediana_(other.mediana_) {}
00026
00027     TestStudent(TestStudent&& other)
00028         : vardas_(std::move(other.vardas_)),
00029           pavarde_(std::move(other.pavarde_)),
00030           pazymiai_(std::move(other.pazymiai_)),
00031           egzaminas_(other.egzaminas_),
00032           vidurkis_(other.vidurkis_),
00033           mediana_(other.mediana_) {
00034         other.vardas_.clear();
00035         other.pavarde_.clear();
00036         other.pazymiai_.clear();
00037         other.egzaminas_ = 0;
00038         other.vidurkis_ = 0.0;
00039         other.mediana_ = 0.0;
00040     }
00041
00042     TestStudent& operator=(const TestStudent& other) {
00043         if (this != &other) {
00044             vardas_ = other.vardas_;
00045             pavarde_ = other.pavarde_;
00046             pazymiai_ = other.pazymiai_;
00047             egzaminas_ = other.egzaminas_;
00048             vidurkis_ = other.vidurkis_;
00049             mediana_ = other.mediana_;
00050         }
00051         return *this;
00052     }
00053
00054     TestStudent& operator=(TestStudent&& other) {
00055         if (this != &other) {

```

```

00056         vardas_ = std::move(other.vardas_);
00057         pavarde_ = std::move(other.pavarde_);
00058         pazymiai_ = std::move(other.pazymiai_);
00059         egzaminas_ = other egzaminas_;
00060         vidurkis_ = other.vidurkis_;
00061         mediana_ = other.mediana_;
00062
00063         other.vardas_.clear();
00064         other.pavarde_.clear();
00065         other.pazymiai_.clear();
00066         other egzaminas_ = 0;
00067         other.vidurkis_ = 0.0;
00068         other.mediana_ = 0.0;
00069     }
00070     return *this;
00071 }
00072
00073 ~TestStudent() = default;
00074
00075 const std::string& vardas() const { return vardas_; }
00076 const std::string& pavarde() const { return pavarde_; }
00077 MyVector<int>& pazymiai() { return pazymiai_; }
00078 const MyVector<int>& pazymiai() const { return pazymiai_; }
00079 int egzaminas() const { return egzaminas_; }
00080 double vidurkis() const { return vidurkis_; }
00081 double mediana() const { return mediana_; }
00082
00083 void setVardas(const std::string& vardas) { vardas_ = vardas; }
00084 void setPavarde(const std::string& pavarde) { pavarde_ = pavarde; }
00085 void setEgzaminas(int egzaminas) { egzaminas_ = egzaminas; }
00086 void setVidurkis(double vidurkis) { vidurkis_ = vidurkis; }
00087 void setMediana(double mediana) { mediana_ = mediana; }
00088
00089 private:
00090     std::string vardas_;
00091     std::string pavarde_;
00092     MyVector<int> pazymiai_;
00093     int egzaminas_ = 0;
00094     double vidurkis_ = 0.0;
00095     double mediana_ = 0.0;
00096 };
00097
00098 std::ostream& operator<<(std::ostream& out, const TestStudent& studentas) {
00099     out << studentas.vardas() << " "
00100         << studentas.pavarde() << " "
00101         << studentas egzaminas() << " "
00102         << studentas.pazymiai().size();
00103
00104     for (int pazymis : studentas.pazymiai()) {
00105         out << " " << pazymis;
00106     }
00107
00108     return out;
00109 }
00110
00111 std::istream& operator>>(std::istream& in, TestStudent& studentas) {
00112     std::string vardas;
00113     std::string pavarde;
00114     int egzaminas = 0;
00115     int pazymiu_kiekis = 0;
00116
00117     if (!(in >> vardas >> pavarde >> egzaminas >> pazymiu_kiekis)) {
00118         return in;
00119     }
00120
00121     studentas.setVardas(vardas);
00122     studentas.setPavarde(pavarde);
00123     studentas.setEgzaminas(egzaminas);
00124     studentas.pazymiai().clear();
00125
00126     for (int i = 0; i < pazymiu_kiekis; ++i) {
00127         int pazymis = 0;
00128         if (!(in >> pazymis)) {
00129             return in;
00130         }
00131         studentas.pazymiai().push_back(pazymis);
00132     }
00133
00134     return in;
00135 }
00136
00137 TestStudent tuscias_studentas() {
00138     return TestStudent("", "", MyVector<int>(), 0, 0.0, 0.0);
00139 }
00140
00141 void uzpildyti_studenta(TestStudent& studentas) {
00142     studentas.setVardas("Jonas");

```

```

00143     studentas.setPavarde("Jonaitis");
00144     studentas.pazymiai().push_back(8);
00145     studentas.pazymiai().push_back(9);
00146     studentas.setEgzaminas(10);
00147     studentas.setVidurkis(9.2);
00148     studentas.setMediana(8.5);
00149 }
00150
00151 void patikrinti_studenta(const TestStudent& studentas) {
00152     assert(studentas vardas() == "Jonas");
00153     assert(studentas.pavarde() == "Jonaitis");
00154     assert(studentas.pazymiai().size() == 2);
00155     assert(studentas.pazymiai()[0] == 8);
00156     assert(studentas.pazymiai()[1] == 9);
00157     assert(studentas.egzaminas() == 10);
00158     assert(studentas.vidurkis() == 9.2);
00159     assert(studentas.mediana() == 8.5);
00160 }
00161
00162 void patikrinti_tuscia_studenta(const TestStudent& studentas) {
00163     assert(studentas.vardas().empty());
00164     assert(studentas.pavarde().empty());
00165     assert(studentas.pazymiai().empty());
00166     assert(studentas.egzaminas() == 0);
00167     assert(studentas.vidurkis() == 0.0);
00168     assert(studentas.mediana() == 0.0);
00169 }
00170
00171 void test_default_constructor() {
00172     MyVector<int> v;
00173
00174     assert(v.empty());
00175     assert(v.size() == 0);
00176     assert(v.capacity() == 1);
00177 }
00178
00179 void test_push_back_and_capacity_growth() {
00180     MyVector<int> v;
00181
00182     v.push_back(10);
00183     assert(v.size() == 1);
00184     assert(v.capacity() == 1);
00185     assert(v[0] == 10);
00186
00187     v.push_back(20);
00188     assert(v.size() == 2);
00189     assert(v.capacity() == 2);
00190     assert(v[1] == 20);
00191
00192     v.push_back(30);
00193     assert(v.size() == 3);
00194     assert(v.capacity() == 4);
00195     assert(v[2] == 30);
00196 }
00197
00198 void test_front_back_index_and_data() {
00199     MyVector<int> v;
00200     v.push_back(4);
00201     v.push_back(5);
00202     v.push_back(6);
00203
00204     assert(v.front() == 4);
00205     assert(v.back() == 6);
00206     assert(v.data()[1] == 5);
00207
00208     v[1] = 99;
00209     assert(v[1] == 99);
00210     assert(v.data()[1] == 99);
00211 }
00212
00213 void test_at_checks_bounds() {
00214     MyVector<int> v;
00215     v.push_back(7);
00216
00217     assert(v.at(0) == 7);
00218
00219     bool thrown = false;
00220     try {
00221         v.at(1);
00222     } catch (const std::out_of_range&) {
00223         thrown = true;
00224     }
00225     assert(thrown);
00226 }
00227
00228 void test_pop_back_and_clear() {
00229     MyVector<int> v;

```



```
00230     v.push_back(1);
00231     v.push_back(2);
00232     const size_t old_capacity = v.capacity();
00233
00234     v.pop_back();
00235     assert(v.size() == 1);
00236     assert(v.back() == 1);
00237
00238     v.pop_back();
00239     v.pop_back();
00240     assert(v.empty());
00241
00242     v.push_back(5);
00243     v.clear();
00244     assert(v.empty());
00245     assert(v.capacity() == old_capacity);
00246 }
00247
00248 void test_copy_constructor_deep_copy() {
00249     MyVector<int> original;
00250     original.push_back(1);
00251     original.push_back(2);
00252
00253     MyVector<int> copy(original);
00254     original[0] = 100;
00255
00256     assert(copy.size() == 2);
00257     assert(copy[0] == 1);
00258     assert(copy[1] == 2);
00259 }
00260
00261 void test_copy_assignment_deep_copy_and_self_assignment() {
00262     MyVector<int> original;
00263     original.push_back(3);
00264     original.push_back(4);
00265
00266     MyVector<int> copy;
00267     copy = original;
00268     original[1] = 400;
00269
00270     assert(copy.size() == 2);
00271     assert(copy[0] == 3);
00272     assert(copy[1] == 4);
00273
00274     MyVector<int>* same = &copy;
00275     copy = *same;
00276     assert(copy.size() == 2);
00277     assert(copy[0] == 3);
00278     assert(copy[1] == 4);
00279 }
00280
00281 void test_move_constructor() {
00282     MyVector<int> original;
00283     original.push_back(5);
00284     original.push_back(6);
00285
00286     MyVector<int> moved(std::move(original));
00287
00288     assert(moved.size() == 2);
00289     assert(moved[0] == 5);
00290     assert(moved[1] == 6);
00291     assert(original.empty());
00292     assert(original.capacity() == 0);
00293 }
00294
00295 void test_move_assignment() {
00296     MyVector<int> original;
00297     original.push_back(7);
00298     original.push_back(8);
00299
00300     MyVector<int> moved;
00301     moved.push_back(100);
00302     moved = std::move(original);
00303
00304     assert(moved.size() == 2);
00305     assert(moved[0] == 7);
00306     assert(moved[1] == 8);
00307     assert(original.empty());
00308     assert(original.capacity() == 0);
00309 }
00310
00311 void test_reserve_resize_and_shrink_to_fit() {
00312     MyVector<int> v;
00313     v.reserve(10);
00314     assert(v.capacity() == 10);
00315     assert(v.size() == 0);
00316 }
```

```

00317     v.resize(3, 42);
00318     assert(v.size() == 3);
00319     assert(v.capacity() == 10);
00320     assert(v[0] == 42);
00321     assert(v[1] == 42);
00322     assert(v[2] == 42);
00323
00324     v.resize(5);
00325     assert(v.size() == 5);
00326     assert(v[3] == 0);
00327     assert(v[4] == 0);
00328
00329     v.resize(2);
00330     assert(v.size() == 2);
00331
00332     v.shrink_to_fit();
00333     assert(v.capacity() == 2);
00334     assert(v[0] == 42);
00335     assert(v[1] == 42);
00336 }
00337
00338 void test_iterators_and_algorithm_support() {
00339     MyVector<int> v;
00340     v.push_back(3);
00341     v.push_back(1);
00342     v.push_back(2);
00343
00344     int sum = 0;
00345     for (int value : v) {
00346         sum += value;
00347     }
00348     assert(sum == 6);
00349
00350     std::sort(v.begin(), v.end());
00351     assert(v[0] == 1);
00352     assert(v[1] == 2);
00353     assert(v[2] == 3);
00354 }
00355
00356 void test_const_iterators() {
00357     MyVector<int> v;
00358     v.push_back(11);
00359     v.push_back(12);
00360
00361     const MyVector<int>& cv = v;
00362     assert(*cv.begin() == 11);
00363     assert(*(cv.end() - 1) == 12);
00364     assert(*cv.cbegin() == 11);
00365     assert(*(cv.cend() - 1) == 12);
00366 }
00367
00368 void test_erase_single_and_range() {
00369     MyVector<int> v;
00370     for (int i = 1; i <= 5; ++i) {
00371         v.push_back(i);
00372     }
00373
00374     auto it = v.erase(v.begin() + 1);
00375     assert(*it == 3);
00376     assert(v.size() == 4);
00377     assert(v[0] == 1);
00378     assert(v[1] == 3);
00379     assert(v[2] == 4);
00380     assert(v[3] == 5);
00381
00382     it = v.erase(v.begin() + 1, v.begin() + 3);
00383     assert(*it == 5);
00384     assert(v.size() == 2);
00385     assert(v[0] == 1);
00386     assert(v[1] == 5);
00387 }
00388
00389 void test_insert_begin_middle_end() {
00390     MyVector<int> v;
00391     v.push_back(2);
00392     v.push_back(4);
00393
00394     auto it = v.insert(v.begin(), 1);
00395     assert(*it == 1);
00396     assert(v[0] == 1);
00397
00398     it = v.insert(v.begin() + 2, 3);
00399     assert(*it == 3);
00400     assert(v[2] == 3);
00401
00402     it = v.insert(v.end(), 5);
00403     assert(*it == 5);

```

```

00404
00405     assert(v.size() == 5);
00406     for (size_t i = 0; i < v.size(); ++i) {
00407         assert(v[i] == static_cast<int>(i + 1));
00408     }
00409 }
00410
00411 void test_assign_and_swap() {
00412     MyVector<int> a;
00413     MyVector<int> b;
00414
00415     a.assign(3, 9);
00416     assert(a.size() == 3);
00417     assert(a[0] == 9);
00418     assert(a[1] == 9);
00419     assert(a[2] == 9);
00420
00421     b.push_back(1);
00422     b.push_back(2);
00423     a.swap(b);
00424
00425     assert(a.size() == 2);
00426     assert(a[0] == 1);
00427     assert(a[1] == 2);
00428     assert(b.size() == 3);
00429     assert(b[0] == 9);
00430 }
00431
00432 void test_student_rule_of_five_copy_constructor() {
00433     TestStudent pirmas = tuscias_students();
00434     uzpildyti_studenta(pirmas);
00435
00436     TestStudent antras(pirmas);
00437     patikrinti_studenta(antras);
00438
00439     pirmas.setVardas("Petras");
00440     pirmas.pazymiai().push_back(5);
00441
00442     assert(antras.vardas() == "Jonas");
00443     assert(antras.pazymiai().size() == 2);
00444 }
00445
00446 void test_student_rule_of_five_move_constructor() {
00447     TestStudent pirmas = tuscias_students();
00448     uzpildyti_studenta(pirmas);
00449
00450     TestStudent antras(std::move(pirmas));
00451     patikrinti_studenta(antras);
00452     patikrinti_tuscia_studenta(pirmas);
00453 }
00454
00455 void test_student_rule_of_five_copy_assignment() {
00456     TestStudent pirmas = tuscias_students();
00457     TestStudent antras = tuscias_students();
00458     uzpildyti_studenta(pirmas);
00459
00460     antras = pirmas;
00461     patikrinti_studenta(antras);
00462
00463     pirmas.setPavarde("Petraitis");
00464     pirmas.pazymiai().clear();
00465
00466     assert(antras.pavarde() == "Jonaitis");
00467     assert(antras.pazymiai().size() == 2);
00468 }
00469
00470 void test_student_rule_of_five_move_assignment() {
00471     TestStudent pirmas = tuscias_students();
00472     TestStudent antras = tuscias_students();
00473     uzpildyti_studenta(pirmas);
00474
00475     antras = std::move(pirmas);
00476     patikrinti_studenta(antras);
00477     patikrinti_tuscia_studenta(pirmas);
00478 }
00479
00480 void test_student_stream_output() {
00481     TestStudent studentas = tuscias_students();
00482     uzpildyti_studenta(studentas);
00483     std::ostringstream out;
00484
00485     out << studentas;
00486
00487     assert(out.str() == "Jonas Jonaitis 10 2 8 9");
00488 }
00489
00490 void test_student_stream_input() {

```

```

00491     TestStudent studentas = tuscias_studentas();
00492     std::istringstream in("Ona Onaite 9 3 10 8 7");
00493
00494     in » studentas;
00495
00496     assert(studentas vardas() == "Ona");
00497     assert(studentas pavarde() == "Onaite");
00498     assert(studentas egzaminas() == 9);
00499     assert(studentas pazymiai().size() == 3);
00500     assert(studentas pazymiai()[0] == 10);
00501     assert(studentas pazymiai()[1] == 8);
00502     assert(studentas pazymiai()[2] == 7);
00503 }
00504
00505 #define RUN_TEST(test_name)      \
00506     do {                          \
00507         test_name();              \
00508     } while (false)              \
00509
00510 int main() {
00511     RUN_TEST(test_default_constructor);
00512     RUN_TEST(test_push_back_and_capacity_growth);
00513     RUN_TEST(test_front_back_index_and_data);
00514     RUN_TEST(test_at_checks_bounds);
00515     RUN_TEST(test_pop_back_and_clear);
00516     RUN_TEST(test_copy_constructor_deep_copy);
00517     RUN_TEST(test_copy_assignment_deep_copy_and_self_assignment);
00518     RUN_TEST(test_move_constructor);
00519     RUN_TEST(test_move_assignment);
00520     RUN_TEST(test_reserve_resize_and_shrink_to_fit);
00521     RUN_TEST(test_iterators_and_algorithm_support);
00522     RUN_TEST(test_const_iterators);
00523     RUN_TEST(test_erase_single_and_range);
00524     RUN_TEST(test_insert_begin_middle_end);
00525     RUN_TEST(test_assign_and_swap);
00526     RUN_TEST(test_student_rule_of_five_copy_constructor);
00527     RUN_TEST(test_student_rule_of_five_move_constructor);
00528     RUN_TEST(test_student_rule_of_five_copy_assignment);
00529     RUN_TEST(test_student_rule_of_five_move_assignment);
00530     RUN_TEST(test_student_stream_output);
00531     RUN_TEST(test_student_stream_input);
00532
00533     std::cout << "Visi MyVector unit testai praejo." << std::endl;
00534     return 0;
00535 }

```

6.7 vector.h

```

00001 #ifndef MY_VECTOR_H
00002 #define MY_VECTOR_H
00003
00004 #include "library.h"
00005 #include <stdexcept>
00006
00011 template <typename T>
00012 class MyVector{
00013     private:
00014         size_t size_;
00015         size_t capacity_;
00016         T* data_;
00020         void grow(){ //padidina capacity_ vektoriui
00021             if(capacity_ == 0) capacity_ = 1;
00022             else capacity_ = capacity_ * 2;
00023             T * newData = new T[capacity_];
00024             for(size_t i=0; i<size_; i++){
00025                 newData[i] = data_[i];
00026             }
00027             delete[] data_;
00028             data_ = newData;
00029         }
00030     public:
00031
00035         using value_type = T;
00036
00037         /*
00038         -----
00039         Konstruktoriai
00040         */
00044         MyVector(): //vektoriaus konstruktorius
00045             size_(0),
00046             capacity_(1),
00047             data_(new T[capacity_]){}
00048
00053         MyVector(const MyVector& other): //copy konstruktorius

```

```

00054         size_(other.size_),
00055         capacity_(other.capacity_),
00056         data_(new T[other.capacity_]){
00057             for(size_t i =0; i< size_; i++){
00058                 data_[i] = other.data_[i];
00059             }
00060         }
00061
00066     MyVector(MyVector&& other): // move konstruktorius
00067         size_(other.size_),
00068         capacity_(other.capacity_),
00069         data_(other.data_)
00070     {
00071         other.size_ = 0;
00072         other.capacity_ = 0;
00073         other.data_ = nullptr;
00074     }
00075
00076     /*
00077     -----
00078     Operatoriai
00079     */
00080
00086     T & operator[](size_t index){ //grazina vektoriaus reiksme duotam indexe
00087         return data_[index];
00088     }
00089
00095     MyVector& operator=(const MyVector& other){ // lygu operatorius :(
00096         if(this != &other){
00097             delete[] data_;
00098             size_ = other.size_;
00099             capacity_ = other.capacity_;
00100             data_ = new T[other.capacity_];{
00101                 for(size_t i =0; i< size_; i++){
00102                     data_[i] = other.data_[i];
00103                 }
00104             }
00105         }
00106         return *this;
00107     }
00108
00114     MyVector& operator=(MyVector&& other) { //move assignment operatorius
00115         if (this != &other) {
00116             delete[] data_;
00117
00118             size_ = other.size_;
00119             capacity_ = other.capacity_;
00120             data_ = other.data_;
00121
00122             other.size_ = 0;
00123             other.capacity_ = 0;
00124             other.data_ = nullptr;
00125         }
00126
00127         return *this;
00128     }
00129
00135     const T& operator[](size_t index) const{
00136         return data_[index];
00137     }
00138
00139     /*
00140     -----
00141     Funkcijos
00142     */
00143
00148     void push_back(const T& val){ //Iraso nauja kintamaji i vektorio funkcija
00149         if (size_ == capacity_) grow();
00150         data_[size_] = val;
00151         size_++;
00152     }
00153
00157     void pop_back(){ //istrina nari vektoriaus gale
00158         if(size_ > 0) size_--;
00159     }
00160
00165     bool empty(){ //patikrina ar vektorius tuscias
00166         return size_ == 0;
00167     }
00168
00173     bool empty() const{ //const versija sios funkcijos
00174         return size_ == 0;
00175     }
00176
00181     size_t size(){ //grazina vektoriaus dydi
00182         return size_;
00183     }

```

```

00184
00189     size_t size() const { //konstanta
00190         return size_;
00191     }
00192
00197     size_t capacity() { //grazina vektoriaus capacity_
00198         return capacity_;
00199     }
00200
00205     size_t capacity() const {
00206         return capacity_;
00207     }
00208
00212     void clear() { //isvalo vektoriu
00213         size_ = 0;
00214     }
00215
00219     void shrink_to_fit() { //sumazina vektoriaus capacity_ iki uzpildyto vektoriaus dydzio
00220         if (capacity_ > size_) {
00221             T* newData = new T[size_];
00222
00223             for (size_t i = 0; i < size_; i++) {
00224                 newData[i] = data_[i];
00225             }
00226
00227             delete[] data_;
00228             data_ = newData;
00229             capacity_ = size_;
00230         }
00231     }
00232
00237     T* begin() {
00238         return data_;
00239     }
00240
00245     T* end() {
00246         return data_ + size_;
00247     }
00248
00253     const T* begin() const {
00254         return data_;
00255     }
00256
00261     const T* end() const {
00262         return data_ + size_;
00263     }
00264
00269     T& back() {
00270         return data_[size_ - 1];
00271     }
00272
00277     const T& back() const {
00278         return data_[size_ - 1];
00279     }
00280
00285     T& front() {
00286         return data_[0];
00287     }
00288
00293     const T& front() const {
00294         return data_[0];
00295     }
00296
00303     T* erase(T* first, T* last) {
00304         std::move(last, end(), first);
00305         size_ -= last - first;
00306         return first;
00307     }
00308
00314     T* erase(T* pos) {
00315         return erase(pos, pos + 1);
00316     }
00317
00324     T& at(size_t index) { //patikrina vektoriaus ribas
00325         if (size_ <= index) {
00326             throw std::out_of_range("MyVector index out of range");
00327         }
00328         return data_[index];
00329     }
00330
00337     const T& at(size_t index) const {
00338         if (index >= size_) {
00339             throw std::out_of_range("MyVector index out of range");
00340         }
00341         return data_[index];
00342     }
00343

```

```

00344
00349 void reserve(size_t newCapacity){ //pakeisti vektoriaus capacity
00350     if(newCapacity <= capacity_){
00351         return;
00352     }
00353
00354     T* newData = new T[newCapacity];
00355     for(size_t i=0; i<size_; i++){
00356         newData[i] = data_[i];
00357     }
00358
00359     delete[] data_;
00360     data_ = newData;
00361     capacity_ = newCapacity;
00362 }
00363
00368 void resize(size_t newSize){ //resizina vektoriu iki tam tikro dydzio
00369     if(newSize > capacity_) reserve(newSize);
00370
00371     for(size_t i=size_; i<newSize; i++){
00372         data_[i] = T();
00373     }
00374
00375     size_ = newSize;
00376 }
00377
00383 void resize(size_t newSize, const T& value) { //resizina vektoriu iki naudotujui reikalingo
dydzio ir tuscias laukus uzpildo duotom reiksmes
00384     if (newSize > capacity_) reserve(newSize);
00385
00386     for (size_t i = size_; i < newSize; i++) {
00387         data_[i] = value;
00388     }
00389
00390     size_ = newSize;
00391 }
00392
00397 T* data(){ //vektoriaus pradzios adresas
00398     return data_;
00399 }
00400
00405 T* cbegin() const{ //vektoriaus pradzios adreas
00406     return data_;
00407 }
00408
00413 const T* cend() const { //bektoriaus pabaigos adresas
00414     return data_ + size_;
00415 }
00416
00421 void swap(MyVector& other) { //apkeicia vektorius nariais, vietoj move ar copy
00422     std::swap(size_, other.size_);
00423     std::swap(capacity_, other.capacity_);
00424     std::swap(data_, other.data_);
00425 }
00426
00432 void assign(size_t count, const T& value){ //priskiria vektoriui reiksme
00433     if(count > capacity_) reserve(count);
00434     for(size_t i = 0; i<count; i++){
00435         data_[i] = value;
00436     }
00437     size_ = count;
00438 }
00439
00446 T* insert(T* pos, const T& value) { //ideda nauja reiksme i nauja indexa vektoriuje
00447     size_t index = pos - begin();
00448
00449     if (size_ == capacity_) grow();
00450
00451     for (size_t i = size_; i > index; i--) {
00452         data_[i] = data_[i - 1];
00453     }
00454
00455     data_[index] = value;
00456     size_++;
00457
00458     return data_ + index;
00459 }
00460
00464 ~MyVector(){ //destruktorius
00465     delete[] data_;
00466     size_ = 0;
00467     capacity_ = 0;
00468 }
00469 };
00470
00477 template <typename Container>
00478 void pildymas(Container& vec, int sk){

```

```
00479     const auto pradzia = std::chrono::high_resolution_clock::now();
00480     for(int i=0; i<sk; i++){
00481         vec.push_back(i);
00482     }
00483     const auto pabaiga = std::chrono::high_resolution_clock::now();
00484     const std::chrono::duration<double, std::milli> trukme = pabaiga - pradzia;
00485     std::cout << trukme.count() << std::endl;
00486 }
00487
00488 #endif
```


Index

- ~MyVector
 - MyVector< T >, [12](#)
- ~Studentas
 - Studentas< GradeContainer >, [21](#)
- ~Zmogus
 - Zmogus, [27](#)
- assign
 - MyVector< T >, [12](#)
- at
 - MyVector< T >, [12](#)
- back
 - MyVector< T >, [13](#)
- begin
 - MyVector< T >, [13](#)
- capacity
 - MyVector< T >, [13](#), [14](#)
- capacity_
 - MyVector< T >, [20](#)
- cbegin
 - MyVector< T >, [14](#)
- cend
 - MyVector< T >, [14](#)
- clear
 - MyVector< T >, [14](#)
- data
 - MyVector< T >, [14](#)
- data_
 - MyVector< T >, [20](#)
- egzaminas
 - Studentas< GradeContainer >, [22](#)
 - TestStudent, [24](#)
- egzaminas_
 - Studentas< GradeContainer >, [23](#)
 - TestStudent, [26](#)
- empty
 - MyVector< T >, [14](#), [15](#)
- end
 - MyVector< T >, [15](#)
- erase
 - MyVector< T >, [15](#)
- front
 - MyVector< T >, [16](#)
- grow
 - MyVector< T >, [16](#)

- insert
 - MyVector< T >, [16](#)
- mediana
 - Studentas< GradeContainer >, [22](#)
 - TestStudent, [24](#)
- mediana_
 - Studentas< GradeContainer >, [23](#)
 - TestStudent, [26](#)
- MyVector
 - MyVector< T >, [11](#)
- MyVector< T >, [9](#)
 - ~MyVector, [12](#)
 - assign, [12](#)
 - at, [12](#)
 - back, [13](#)
 - begin, [13](#)
 - capacity, [13](#), [14](#)
 - capacity_, [20](#)
 - cbegin, [14](#)
 - cend, [14](#)
 - clear, [14](#)
 - data, [14](#)
 - data_, [20](#)
 - empty, [14](#), [15](#)
 - end, [15](#)
 - erase, [15](#)
 - front, [16](#)
 - grow, [16](#)
 - insert, [16](#)
 - MyVector, [11](#)
 - operator=, [17](#)
 - operator[], [17](#)
 - pop_back, [18](#)
 - push_back, [18](#)
 - reserve, [18](#)
 - resize, [18](#), [19](#)
 - shrink_to_fit, [19](#)
 - size, [19](#)
 - size_, [20](#)
 - swap, [19](#)
 - value_type, [11](#)
- operator=
 - MyVector< T >, [17](#)
 - Studentas< GradeContainer >, [22](#)
 - TestStudent, [24](#), [25](#)
 - Zmogus, [27](#)
- operator[]
 - MyVector< T >, [17](#)

pavarde
 TestStudent, 25
 Zmogus, 27
 pavarde_
 TestStudent, 26
 Zmogus, 28
 pazymiai
 Studentas< GradeContainer >, 22
 TestStudent, 25
 pazymiai_
 Studentas< GradeContainer >, 23
 TestStudent, 26
 pop_back
 MyVector< T >, 18
 push_back
 MyVector< T >, 18

 README, 1
 reserve
 MyVector< T >, 18
 resize
 MyVector< T >, 18, 19

 setEgzaminas
 Studentas< GradeContainer >, 22
 TestStudent, 25
 setMediana
 Studentas< GradeContainer >, 22
 TestStudent, 25
 setPavarde
 TestStudent, 25
 Zmogus, 27
 setVardas
 TestStudent, 25
 Zmogus, 27
 setVidurkis
 Studentas< GradeContainer >, 22
 TestStudent, 25
 shrink_to_fit
 MyVector< T >, 19
 size
 MyVector< T >, 19
 size_
 MyVector< T >, 20
 Studentas
 Studentas< GradeContainer >, 21
 Studentas< GradeContainer >, 20
 ~Studentas, 21
 egzaminas, 22
 egzaminas_, 23
 mediana, 22
 mediana_, 23
 operator=, 22
 pazymiai, 22
 pazymiai_, 23
 setEgzaminas, 22
 setMediana, 22
 setVidurkis, 22
 Studentas, 21
 tipas, 22
 vidurkis, 23
 vidurkis_, 23
 swap
 MyVector< T >, 19

 TestStudent, 23
 egzaminas, 24
 egzaminas_, 26
 mediana, 24
 mediana_, 26
 operator=, 24, 25
 pavarde, 25
 pavarde_, 26
 pazymiai, 25
 pazymiai_, 26
 setEgzaminas, 25
 setMediana, 25
 setPavarde, 25
 setVardas, 25
 setVidurkis, 25
 TestStudent, 24
 vardas, 25
 vardas_, 26
 vidurkis, 25
 vidurkis_, 26
 tipas
 Studentas< GradeContainer >, 22

 value_type
 MyVector< T >, 11
 vardas
 TestStudent, 25
 Zmogus, 28
 vardas_
 TestStudent, 26
 Zmogus, 28
 vidurkis
 Studentas< GradeContainer >, 23
 TestStudent, 25
 vidurkis_
 Studentas< GradeContainer >, 23
 TestStudent, 26

 Zmogus, 26
 ~Zmogus, 27
 operator=, 27
 pavarde, 27
 pavarde_, 28
 setPavarde, 27
 setVardas, 27
 vardas, 28
 vardas_, 28
 Zmogus, 27